



LoRaIM (LoRa Instant Messenger)

Autor: Víctor Manuel Jiménez Sánchez

Técnico Superior en Administración de Sistemas Informáticos en Red (ASIR), 2º Curso

Índice

1. Introducción
2. Objetivos
 - Objetivo principal
 - Objetivos secundarios
3. Marco Teórico
 - Tecnología LoRa y redes LPWAN
 - Ventajas y limitaciones de LoRa para mensajería instantánea
4. Desarrollo del Proyecto
 - Diseño de la arquitectura del sistema
 - Implementación de hardware y software
 - Pruebas realizadas
5. Conclusiones
 - Análisis de resultados
 - Posibles mejoras futuras
6. Anexos
 - Manual de Instalación
 - Manual de Usuario
 - Código comentado
7. Bibliografía

1. Introducción

LoRaIM (LoRa Instant Messenger) es un proyecto de mensajería instantánea que utiliza la tecnología de comunicación inalámbrica **LoRa** para permitir el intercambio de mensajes de texto entre usuarios sin necesidad de infraestructura convencional (sin cobertura celular, sin WiFi ni Internet). El propósito de este proyecto es desarrollar un sistema de bajo costo y bajo consumo energético que ofrezca una alternativa de comunicación en zonas aisladas o en situaciones de emergencia. Las redes basadas en LoRa permiten interconectar enlaces inalámbricos a largas distancias, llegando a comunicar pueblos y comunidades rurales con centros urbanos cercanos. A diferencia de los servicios de mensajería tradicionales que dependen de redes móviles o de Internet, LoRaIM funciona **off-grid**, es decir, crea su propia red de comunicación independiente. Esto resulta especialmente útil cuando no hay infraestructura disponible – por ejemplo, en áreas rurales sin cobertura telefónica, en campamentos al aire libre o tras desastres naturales donde las redes convencionales estén caídas. De hecho, las comunicaciones mediante dispositivos LoRa pueden ser muy valiosas cuando se interrumpe el servicio celular, extendiendo la conectividad más allá del alcance de las torres de telefonía.

En el contexto académico, este proyecto combina conocimientos de redes, comunicaciones inalámbricas e Internet de las Cosas (IoT), desarrollo en frontend, backend, web dinámica (web app), desarrollo micropython para diseñar y desplegar un sistema de mensajería funcional.

A lo largo de esta documentación se describirá detalladamente el desarrollo de LoRaIM: sus objetivos, los fundamentos teóricos, el diseño y la implementación tanto de software como de hardware, las pruebas realizadas, así como las conclusiones obtenidas.

Asimismo, se incluyen anexos con un manual de usuario y un manual de instalación, para facilitar el uso y la replicación del sistema por terceros.

2. Objetivos

Objetivos principales:

- **Desarrollar un sistema de mensajería instantánea independiente (denominado *LoRaIM*) que permita la comunicación de texto entre dos o más usuarios a través de enlaces de radio LoRa de largo alcance, sin necesidad de infraestructura de comunicaciones existente (Internet, redes móviles o WiFi convencionales).**
- **Este proyecto es viable para usarlo en zonas rurales o en países en vías de desarrollo que no cuenten con una estructura de comunicaciones convencional. Este proyecto podría establecerse como un sistema de comunicación económico y de rápida implantación.**

Objetivos secundarios:

- **Aprendizaje e implementación de LoRa:** Investigar los fundamentos de la tecnología LoRa y aplicarlos en la configuración de una red inalámbrica de baja potencia y largo alcance.
- **Despliegue en contenedores Docker con Docker Compose:** Facilitar la portabilidad, replicabilidad y administración del sistema mediante la implementación del servidor web Flask (Frontend), Fastapi (Backend), y Mosquitto (MQTT) en contenedores Docker, utilizando Docker Compose para definir la configuración de los servicios. Este objetivo busca simplificar el despliegue en múltiples nodos y entornos

diversos, garantizando uniformidad en la ejecución y reduciendo problemas relacionados con dependencias o configuración específica del sistema operativo.

- **Diseño de arquitectura ad-hoc:** Diseñar una arquitectura de sistema **peer-to-peer** o de red ad-hoc capaz de interconectar los dispositivos de los usuarios usando módulos LoRa.
- **Desarrollo de software en Python:** Implementar la lógica de la aplicación en Python, incluyendo la interfaz de usuario (ej. una aplicación web local) y la integración con los módulos de radio LoRa mediante bibliotecas o controladores adecuados.
- **Integración de hardware y comunicaciones:** Montar los componentes de hardware necesarios: microcontroladores o microordenadores (como Raspberry Pi), módulos de radio LoRa (transceptores de largo alcance). Asegurar que el hardware esté correctamente interconectado y configurado. Para este proyecto utilizaremos específicamente las placas de desarrollo Heltec Lora 32 v3, que son placas programables en python con un procesador ESP32.
- **Pruebas de funcionamiento y alcance:** Realizar pruebas exhaustivas del sistema, comprobando que se pueden enviar y recibir mensajes correctamente entre los dispositivos. Se creará un firmware específico para cargar en un módulo para realizar test dentro de la propia red lora.

Con estos objetivos, el proyecto LoRaIM busca tanto lograr un resultado práctico –un **mensajero instantáneo funcional sobre LoRa**.

3. Marco Teórico

En este apartado se presentan los fundamentos teóricos en los que se apoya el proyecto **LoRaIM**, abarcando las tecnologías y conceptos clave: la tecnología de comunicación LoRa, las características de las redes inalámbricas de baja potencia y largo alcance (LPWAN), y los principios de las comunicaciones punto a punto y en malla que podrían usarse en un sistema de mensajería off-grid.

3.1 Tecnología LoRa y redes LPWAN

LoRa (del inglés *Long Range*, “largo alcance”) es una tecnología de comunicaciones inalámbricas desarrollada específicamente para lograr **comunicaciones a larga distancia con bajo consumo de energía**.

Técnicamente, LoRa define una modulación de radio basada en *Chirp Spread Spectrum* (CSS), en la cual la información se codifica en pulsos tipo “chirp” de frecuencia variable. Esta modulación de espectro ensanchado proporciona gran robustez frente al ruido e interferencias, permitiendo que las señales LoRa puedan ser recibidas a kilómetros de distancia incluso con potencia de transmisión muy baja. En otras palabras, LoRa fue diseñada para **alcanzar distancias muy superiores** a las de tecnologías inalámbricas convencionales de corto alcance como WiFi, Bluetooth o ZigBee, a costa de transmitir datos a velocidades más reducidas.

LoRa se engloba dentro de las llamadas redes de área amplia y baja potencia (LPWAN, *Low Power Wide Area Network*). Es la capa física de comunicación: especifica cómo se modulan y envían las ondas de radio. Las frecuencias de operación de LoRa se sitúan en bandas libres de licencia sub-GHz (bandas ISM). En Europa típicamente se usa la banda EU868 (868 MHz), mientras que en América se usan 915 MHz, y también existen variantes en 433 MHz, 923 MHz, etc., dependiendo de la región. El uso de frecuencias sub-GHz contribuye al largo alcance, ya que estas frecuencias se propagan mejor y atraviesan obstáculos más efectivamente que las bandas más altas. Asimismo, LoRa puede también operar en 2.4 GHz (existe hardware LoRa a 2.4 GHz), logrando mayores *bitrate* a costa de menor alcance, aunque el uso más extendido son las bandas inferiores por su amplio radio de cobertura.

Una característica importante de LoRa es su adaptabilidad en términos de alcance vs. velocidad de datos. Emplea diferentes factores de ensanchamiento o *Spreading Factors* (SF). Un SF más alto significa que cada bit de datos se “espacia” más en el tiempo (chirps más largos), lo cual **aumenta el alcance y la sensibilidad de recepción** pero reduce la tasa de bits. Por ejemplo, con LoRa pueden alcanzarse **tasas de datos desde ~0.3 kbps hasta 50 kbps** como máximo, dependiendo de la configuración. En condiciones ideales de línea de vista (LoS) y con SF óptimos, es posible comunicar dispositivos LoRa a distancias del orden de **10 a 20 kilómetros**. En entornos urbanos con obstáculos la distancia típica es menor (varios kilómetros o menos, según densidad de edificios), pero sigue siendo muy superior a la de WiFi/Bluetooth. Esta capacidad de comunicación a larga distancia con baja potencia hace a LoRa especialmente atractiva para **sensores IoT alimentados con batería**, donde pequeñas ráfagas de datos (temperatura, posición, etc.) deben enviarse esporádicamente a un receptor lejano.

Es importante distinguir **LoRa** de **LoRaWAN**. LoRa es la tecnología de radio (capa física), mientras que **LoRaWAN** es un protocolo de red construido sobre LoRa. En concreto, LoRaWAN especifica el **protocolo de enlace de datos y la arquitectura de red** para gestionar la comunicación entre dispositivos LoRa a gran escala (normalmente en topología estrella a través de **gateways** o puertas de enlace). LoRaWAN define cómo se estructuran los paquetes, cómo se accede al medio (por ejemplo, clases A, B, C de dispositivos con distintas estrategias de recepción continua o programada) y cómo se integran las comunicaciones LoRa en Internet (los gateways LoRaWAN envían los datos a servidores en la nube). En suma, **LoRaWAN es un estándar de capa MAC/red** promovido por la LoRa Alliance para redes LPWAN amplias, pensado para IoT masivo, con características como seguridad extremo a extremo, gestión de duplicados, etc.

Cabe aclarar que **LoRaIM no utiliza directamente LoRaWAN**, ya que nuestro caso de uso no involucra una red pública con gateways ni una infraestructura centralizada. En lugar de ello, LoRaIM se basa en la comunicación **LoRa en modo punto a punto (P2P)** entre nodos. Esto significa que los transceptores LoRa en nuestros dispositivos se comunican directamente entre sí, sin pasar por gateways intermedios ni servidores de red. LoRa permite este modo P2P configurando manualmente parámetros comunes en todos los nodos (frecuencia, SF, códigos de corrección de error, etc.), de forma que cualquier paquete transmitido por un nodo pueda ser demodulado por los demás en rango. Algunos módulos comerciales (como los de Reyax) implementan además un esquema básico de direccionamiento en P2P: cada módulo puede tener una dirección ID, y se pueden enviar paquetes a un ID específico, lo que facilita construir un **protocolo simple de mensajería** sin colisiones. Esto resulta suficiente para un messenger de pocos usuarios; para redes más grandes o de topología diversa, habría que considerar protocolos más complejos.

3.2 Ventajas y limitaciones de LoRa para mensajería instantánea

LoRa presenta **grandes ventajas** en escenarios donde otras tecnologías fallan o son costosas. Ya hemos mencionado su amplio alcance y bajo consumo. Además, opera en bandas libres, por lo que **no requiere pagar por datos ni licencias** de espectro; esto permite que comunidades o particulares monten sus propias redes LoRaIM sin incurrir en gastos recurrentes. En contextos rurales o de países en desarrollo donde la cobertura celular es escasa, soluciones basadas en LoRa pueden proveer servicios de comunicación básicos (mensajes, alertas) a una fracción del coste de desplegar antenas celulares. Por ejemplo, en México se estima que un 16% de la población (más de 19 millones de personas) vive sin cobertura celular – para estas comunidades, sistemas como LoRaIM podrían brindar un medio de comunicación esencial. Otra aplicación evidente es en **situaciones de desastre o emergencia**, donde las redes tradicionales se caen: con un mensajero LoRa los equipos de rescate o habitantes podrían seguir comunicándose localmente para coordinar, siempre que tengan dispositivos LoRaIM a mano.

Sin embargo, es importante reconocer las **limitaciones** de LoRa, especialmente al intentar usarlo para mensajería instantánea (pensada originalmente para seres humanos, no solo sensores). La principal limitación es el **ancho de banda extremadamente bajo** comparado con estándares como WiFi o redes móviles. En LoRaIM solo es práctico enviar texto plano u otros datos muy pequeños; no es viable transferir archivos grandes, imágenes o audio en tiempo real. Incluso los mensajes de texto tienen restricciones de tamaño: los paquetes LoRa suelen admitir en torno a 50-255 bytes útiles como máximo (dependiendo de la configuración). Si un usuario intentara enviar un párrafo muy largo, habría que fragmentarlo en múltiples paquetes, aumentando la latencia.

Relacionado con lo anterior, está la restricción regulatoria del **ciclo de trabajo (duty cycle)** en las bandas ISM. En Europa, por ejemplo, la banda 868 MHz limita a un 1% el tiempo máximo de transmisión de un dispositivo (por normativa ETSI). Esto significa que un nodo LoRa no puede estar transmitiendo constantemente; después de enviar un paquete debe permanecer en silencio la mayor parte del tiempo. Para LoRaIM, implica que la frecuencia de mensajes es baja – no es para chats continuos tipo WhatsApp, sino para mensajes esporádicos. Un envío típico de LoRa (varios bytes a SF alto) podría tardar del orden de 0.5 a 2 segundos en el aire; tras eso, legalmente el dispositivo debería esperar quizá 1-2 minutos antes de otra transmisión larga, para cumplir el 1%. En la práctica, configuraciones de LoRaIM deben balancear el **SF (Spreading Factor)** para no exceder estos límites y quizá implementar mecanismos de espera si el usuario intenta enviar muchos mensajes seguidos.

Otra limitación es la **latencia e interfaz**: LoRa no establece una conexión persistente ni garantiza entrega inmediata. No hay confirmaciones automáticas estilo TCP a menos que se programe. Para que LoRaIM se asemeje a una mensajería “instantánea”, se pueden integrar ACKs (confirmaciones) a nivel de aplicación – por ejemplo, que el nodo receptor envíe de vuelta un pequeño paquete confirmando que recibió el mensaje, para notificar al remitente. Esto agrega algo de tráfico pero mejora la fiabilidad percibida. Aun así, comparado con sistemas en línea, habrá mayores demoras e incertidumbre en la recepción.

Por último, está la cuestión de la **escalabilidad y topología**. LoRaIM en su versión básica conecta dos nodos directamente. Si quisiéramos ampliar a una red con múltiples usuarios distribuidos (más de 2 nodos), tendríamos que enfrentar desafíos de encaminamiento (routing) o **red mallada (mesh)**. LoRa en sí soporta la posibilidad de redes tipo mesh (no definidas por LoRaWAN estándar, pero implementables a nivel de aplicación) – de hecho, existen proyectos comunitarios donde varios nodos LoRa retransmiten mensajes formando una malla inundada. Esto permite extender la cobertura: cada nodo actúa como repetidor, pasando el mensaje hasta alcanzar al destinatario final, a costa de mayor complejidad y consumo. En LoRaIM podríamos concebir una futura extensión para modo mesh (como el proyecto *Meshtastic* o *LoRa Messenger* de Hackaday, que crean chats off-grid multi-salto). En esta primera versión nos centramos en enlaces directos; no obstante, el diseño deja abierta la puerta para evolución hacia mesh, dado que LoRa **no requiere infraestructura fija** y es plausible implementar repetidores. Hay que considerar que en un mesh LoRa la latencia aumenta con cada salto y el riesgo de colisiones también, por lo que protocolos de enrutamiento inteligentes serían necesarios.

En resumen, desde el punto de vista teórico, LoRa ofrece los cimientos ideales para comunicaciones en ausencia de red: largo alcance, baja potencia, operación libre y sencilla. La contrapartida son las limitaciones en velocidad y capacidad, que obligan a optimizar el uso (mensajes cortos, esenciales) y a manejar expectativas de desempeño. **LoRaIM se apoya en estos fundamentos**, aprovechando las ventajas (comunicación a kilómetros sin coste) y mitigando las limitaciones mediante diseño de protocolo simple y buenas prácticas (como limitar el tamaño y frecuencia de mensajes, y confirmar la recepción).

4. Desarrollo del Proyecto

A continuación, se detalla el desarrollo práctico de LoRaIM, incluyendo el **diseño de la arquitectura** del sistema y la **implementación** concreta de hardware y software. Se describe cómo se han materializado los objetivos propuestos, qué componentes forman el sistema, cómo interactúan entre sí, el entorno de desarrollo, las herramientas utilizadas, así como las pruebas realizadas durante la construcción del mensajero LoRa.

4.1 Diseño

En la fase de diseño se definió la **arquitectura general** de LoRaIM, abarcando tanto la estructura física (dispositivos y conexiones) como la estructura lógica (módulos de software y flujo de datos). La solución adoptada busca un equilibrio entre funcionalidad, simplicidad y robustez, teniendo en cuenta las peculiaridades de LoRa discutidas en el marco teórico.

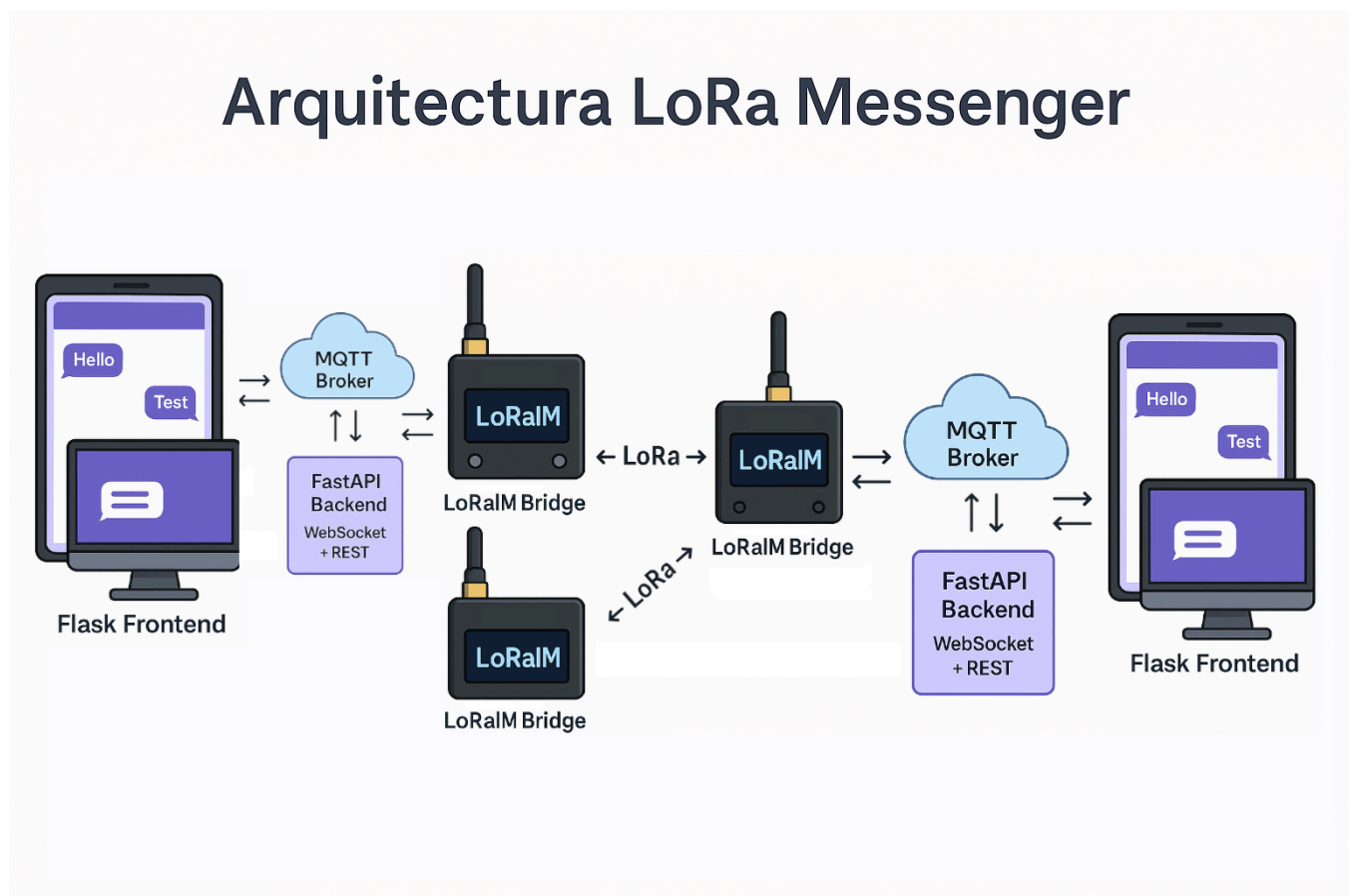


Figura 1: Arquitectura de red propuesta para LoRaIM.

Figura 1: Diagrama general de la arquitectura de LoRaIM. En la arquitectura propuesta, los **nodos LoRa** (basados en placas Heltec WiFi LoRa 32 V3) se comunican entre sí mediante radio LoRa, mientras que uno de ellos actúa como **punto de acceso** hacia una red IP. Este **LoRaIM Bridge** se conecta a una red WiFi local y a un **broker MQTT** (servidor de mensajería ligera) que corre en un servidor o computadora central. Sobre este mismo servidor se ejecuta un backend desarrollado en **FastAPI** (que ofrece una API REST y un servicio WebSocket) y un frontend web (implementado con **Flask**) que sirve una interfaz de chat al usuario final. En el diagrama se observa cómo un mensaje puede viajar desde un dispositivo LoRa remoto hasta la pantalla de un usuario en el navegador: el nodo puente retransmite vía MQTT los datos recibidos por radio, el backend los procesa y los entrega al frontend en tiempo real mediante WebSockets. Del mismo modo, cualquier mensaje enviado desde la interfaz web se publica por MQTT y es capturado por el nodo puente, que lo emite por LoRa hacia los demás nodos.

Arquitectura hardware: Los dispositivos empleados son placas **Heltec WiFi LoRa 32 (V3)**, que integran un microcontrolador ESP32-S3 con conectividad WiFi/Bluetooth, un transceptor LoRa **Semtech SX1262** (operando, en nuestro caso, en la banda ISM de 868 MHz) y una pequeña pantalla OLED de 0,96" (128×64 píxeles). Cada nodo Heltec V3 puede funcionar con MicroPython, lo que facilita un desarrollo rápido en comparación con lenguajes de más bajo nivel. Para el despliegue de LoRaLM se consideran al menos dos nodos: uno configurado como **Puente LoRaLM (Bridge)** con capacidad WiFi y MQTT, y uno o más **nodos de prueba** que solo usan LoRa (simulando dispositivos remotos con los que chatear). El puente y los nodos comparten los mismos parámetros de radio (frecuencia, ancho de banda, factor de espalhamento – *spreading factor*, código de corrección de errores, etc.) para poder intercambiar paquetes correctamente. La elección de la placa Heltec se justifica por su integración "todo en uno": el ESP32 proporciona potencia de cómputo y conectividad IP, el módulo SX1262 ofrece enlace de larga distancia, y la OLED permite retroalimentación local para depuración y demostración (mostrando, por ejemplo, el estado de conexión o los mensajes transmitidos).

Arquitectura de software: El sistema se compone de varios subsistemas comunicados entre sí de forma asincrónica:

- **Broker MQTT:** Se utiliza **Eclipse Mosquitto** como broker MQTT central. MQTT (Message Queuing Telemetry Transport) es un protocolo ligero pub/sub ampliamente usado en IoT [sabuhish.github.io](https://github.com/sabuhish). Aquí, actúa como **cola de mensajes** intermediaria entre el mundo LoRa y el mundo web. Se definieron dos *topics* principales: `lorachat/up` para mensajes que viajan *desde* los nodos LoRa hacia el servidor, y `lorachat/down` para mensajes que viajan *desde* el servidor hacia los nodos. Todos los nodos LoRaLM Bridge se suscriben al tópico `lorachat/down` (para recibir mensajes entrantes que deban ser enviados por radio), y publican en `lorachat/up` los paquetes recibidos por LoRa. El backend igualmente se suscribe a `lorachat/up` (para enterarse de nuevos mensajes desde LoRa) y publica en `lorachat/down` lo que el usuario envía desde la web. Esta separación de canales facilita la lógica y permite, por ejemplo, que múltiples nodos puente coexistan (cada uno retransmitiendo a "sus" dispositivos) compartiendo el mismo broker central.
- **Backend FastAPI:** Es el servidor de aplicaciones que coordina la lógica entre MQTT y la web. Fue diseñado siguiendo el patrón **API RESTful** para operaciones clásicas (consulta de últimos mensajes, envío de nuevo mensaje) y también proporciona un **WebSocket** para *push* de mensajes en tiempo real. FastAPI fue elegido por su eficiencia y soporte asíncrono nativo, lo que permite manejar el WebSocket y la comunicación MQTT en paralelo. Al iniciar, el backend establece conexión con el broker MQTT (usando la librería Paho MQTT) y se suscribe a `lorachat/up`. Cada vez que llega un mensaje en ese tópico, el backend lo procesa y lo almacena en una estructura en memoria (una lista o cola de mensajes recientes). Acto seguido, distribuye el mensaje a todos los clientes web conectados mediante el WebSocket (notificando instantáneamente la llegada de un nuevo mensaje). Asimismo, el backend expone un endpoint POST `/publish/` que recibe mensajes desde el frontend; cuando el usuario envía un texto, la API lo publica en `lorachat/down` (para que lo recojan los nodos LoRa) y también lo agrega al historial local con un marcador especial indicando que fue "enviado" desde el lado web. Para permitir que el frontend obtenga el historial al cargarse, existe un endpoint GET `/messages/` que devuelve los últimos *N* mensajes almacenados. La comunicación con MQTT se realiza en un **subproceso** separado (hilo de la librería Paho) que invoca callbacks al recibir mensajes; estos callbacks están sincronizados con la lógica del servidor para actualizar la lista de mensajes y notificar por WebSocket. Se implementó **CORS** abierto en FastAPI para que el frontend (que puede servirse en distinto puerto) pudiera consultar la API sin problemas de origen cruzado.

- **Frontend web (Flask):** La interfaz de usuario es una aplicación web sencilla construida con **Flask** (microframework en Python) que sirve una página de chat interactiva. La decisión de usar Flask obedece a la simplicidad de requerimientos: solo se necesita servir un HTML estático con algo de JavaScript y actuar como proxy hacia la API. El frontend corre típicamente en el mismo servidor (o contenedor Docker) que el backend para mayor simplicidad, aunque en nuestro caso podría haberse unificado con FastAPI; se optó por separarlo para mantener desacopladas la lógica de negocio (backend) y la presentación (frontend). La página web (`index.html`) contiene un sencillo **cliente de chat** escrito en JavaScript. Al cargarse, este cliente hace una petición AJAX (`GET /messages`) para obtener los últimos mensajes y mostrarlos en la ventana de chat, permitiendo al usuario ponerse al día con la conversación. Luego establece una conexión **WebSocket** con el backend (`ws://<servidor>:8000/ws`) para suscribirse a nuevos mensajes entrantes en tiempo real. Cada vez que el backend envía un mensaje por el WebSocket (en formato JSON, con campos `source` y `payload`), el cliente JavaScript lo incorpora al historial visible: los mensajes provenientes de nodos LoRa aparecen con la etiqueta del nombre de origen (por ejemplo, "LoRaTester" o un identificador de nodo), mientras que los mensajes enviados desde la propia interfaz se marcan como **"Yo"** para el usuario local. Este comportamiento se logra asignando la cadena especial `"sent"` como `source` a todos los mensajes que el propio servidor publica a través de la API; el código del frontend reconoce `"sent"` como un mensaje local y muestra "Yo". Adicionalmente, el frontend implementa pequeñas mejoras de usabilidad: notificación de *online/offline* (un indicador en la cabecera que se ilumina cuando el WebSocket está conectado, dando confianza de que se está en línea con el servidor) y un contador de mensajes no leídos (un icono que aparece si llegan mensajes mientras el usuario tiene el scroll del chat hacia arriba). Para enviar un mensaje, el usuario simplemente escribe texto y pulsa el botón "Enviar" (o la combinación Ctrl+Enter); esto realiza una llamada POST via AJAX a `/publish` en el frontend Flask, el cual a su vez reenvía al backend FastAPI. En resumen, el frontend brinda una experiencia similar a la de cualquier aplicación de mensajería, ocultando al usuario final la complejidad de la red LoRa subyacente.
- **Firmware de los nodos LoRa:** Cada nodo Heltec V3 ejecuta un firmware en **MicroPython** adaptado a su rol específico dentro de la red. En particular, se desarrollaron dos variantes principales de firmware:
 - **LoRaIM Bridge (Puente):** es el firmware destinado al nodo puente con WiFi. Su diseño modular lee la configuración desde un archivo `.env` en la flash del dispositivo para parámetros como credenciales WiFi (SSID/contraseña), la IP/dirección del broker MQTT, puertos, tópicos MQTT, y ajustes de radio (frecuencia, SF, potencia, etc.). Al arrancar, el Bridge se conecta automáticamente a la WiFi configurada y luego al broker MQTT usando un cliente ligero (basado en la librería `umqtt.simple` de MicroPython, ligeramente adaptada como `simple.py`). Este cliente MQTT en el nodo se suscribe al tópico `Lorachat/down` para recibir mensajes descendentes. Cuando llega un mensaje MQTT, el firmware lo formatea en JSON (añadiendo su propio identificador de nodo) y lo transmite por LoRa a través del módulo SX1262. Del mismo modo, el Bridge mantiene el transceptor LoRa en escucha continua; en cuanto se recibe un paquete LoRa válido, se publica su contenido en el tópico `Lorachat/up` hacia el broker. Para evitar pérdida de datos en caso de desconexión, el nodo Bridge implementa mecanismos de **robustez**: tiene un **buffer de mensajes pendientes** que almacena temporalmente cualquier mensaje entrante por LoRa si el broker MQTT no está disponible en ese momento, retransmitiéndolos cuando la conexión se restablece. También usa las funcionalidades QoS=1 de MQTT (entrega al menos una vez) y un *keep-alive* de 30 segundos para detectar caídas de enlace. El Bridge configura un **Last Will** (última voluntad) en el broker, de modo que si se desconecta abruptamente el broker publique un aviso de *"offline"* en nombre del nodo (esto alerta a los sistemas de monitoreo que el nodo puente ha caído). Gracias a un algoritmo de

reconexión con *back-off* exponencial, el firmware reintenta la conexión al broker de forma indefinida, aumentando gradualmente el intervalo hasta un máximo (por ejemplo, 30 segundos) para no saturar la red en caso de caídas prolongadas. En cuanto al interfaz local, el Bridge aprovecha la pantalla OLED para mostrar información útil al operario o desarrollador: muestra la dirección IP obtenida en WiFi, el estado de la conexión MQTT (p. ej. *"MQTT: Connected"* o *"MQTT: Offline"*), así como un contador de mensajes enviados (TX) y recibidos (RX) vía LoRa. Cada mensaje transmitido o recibido provoca una actualización en pantalla, facilitando la **visualización en tiempo real** de la actividad radio/MQTT sin necesidad de consola serial. Además, el Bridge genera un identificador único **NODE_NAME** para sí mismo (con formato `"Node-xxxxxx"`) a partir de un hash de la MAC u otro valor único, de tal manera que cada Bridge desplegado podría ser distinguido (por ejemplo, *"Node-A1B2C3"*). Este nombre se inserta en los JSON de los mensajes LoRa emitidos, permitiendo identificar el origen de un mensaje en la aplicación (útil si hubiera múltiples nodos puente cubriendo distintas áreas). En resumen, el firmware Bridge es el corazón que traduce entre el mundo LoRa y el mundo IP, asegurando confiabilidad en la entrega de mensajes y proporcionando información de estado.

- o **Nodos de Prueba (LoRaIM Test Nodes):** son los firmwares para dispositivos que **no** tienen conectividad a Internet, es decir, los nodos terminales que solo se comunican por LoRa. Para probar el sistema se implementaron versiones de firmware que envían y reciben mensajes de forma autónoma, simulando el comportamiento de un usuario remoto. Al iniciar, un nodo de prueba muestra en la OLED un título como *"LoRa Tester"* para indicar que está activo. Estos nodos están programados para **transmitir periódicamente** un mensaje *Ping* de prueba cada cierto intervalo (por ejemplo, cada 30 segundos o 100 segundos según la versión de firmware). El contenido de estos *pings* es un texto corto, a veces formateado como JSON. En la versión más reciente del firmware de prueba, el mensaje se envía como JSON del tipo: `{"from": "LoRaTester", "message": "Ping <N>"}`, donde `<N>` es un contador incremental. El uso de JSON aquí permite que el backend reciba también el campo `from` (por ejemplo *"LoRaTester"*) y lo muestre en la interfaz en lugar de marcar el mensaje como desconocido; de esta forma probamos la identificación de remitentes. El nodo de prueba también está constantemente escuchando por LoRa. Cuando recibe un paquete, su firmware **procesa el contenido**: si el texto recibido es un JSON con los campos esperados, extrae el nombre de origen y el mensaje; si es texto plano, lo toma tal cual. En ambos casos, inmediatamente muestra en la pantalla OLED una línea con el mensaje recibido junto al identificador del emisor. Por ejemplo, podría mostrar: *"RX 0: Node-AB12CF: Hola"* para indicar *Recepción #0* desde el nodo *Node-AB12CF* con el mensaje *"Hola"*. En la línea siguiente de la OLED se presenta la intensidad de señal de esa recepción (RSSI) y la relación señal-ruido (SNR), por ejemplo: *"RSSI -70.5 SNR 9.0"*, lo cual brinda información de calidad del enlace en esa prueba. Además de mostrarlo, el nodo de prueba implementa una sencilla política de **ACK**: tras recibir un mensaje, si las condiciones de duty cycle lo permiten (esto es, si ha pasado el tiempo suficiente desde su última transmisión para no violar restricciones de espectro), responde automáticamente enviando un acuse de recibo. El **ACK** es enviado actualmente como un pequeño mensaje de texto (*"ACK:"*) donde `<M>` es un número que el nodo asigna secuencialmente (por ejemplo *"ACK:1"*, *"ACK:2"*, etc.). Este número corresponde al contador local de transmisiones del nodo de prueba, y aunque no refiere al ID del mensaje original, sirve como confirmación visual de que el nodo respondió *. Al igual que en el Bridge, la pantalla OLED del nodo muestra cada transmisión que realiza, con un mensaje del tipo *"TX 1: Ping 1"* o *"TX 2: ACK:2"*, incluyendo un contador interno de TX para fines de depuración. El firmware de prueba también permite **interacción manual**: el dispositivo está configurado para detectar pulsaciones de un botón físico (el botón usuario en la Heltec, GPIO0) de modo que, si el usuario lo presiona, el nodo

enviará inmediatamente un mensaje (saltándose la espera del intervalo). Esto se ideó para facilitar pruebas en vivo, pudiendo enviar *pings* bajo demanda sin esperar el temporizador. En suma, los nodos de prueba actúan como “usuarios remotos” en nuestro sistema: generan tráfico LoRa representativo (pings), responden a los mensajes que llegan (acks) y muestran localmente la información relevante, demostrando así la eficacia del puente LoRaIM en conectar ambos extremos.

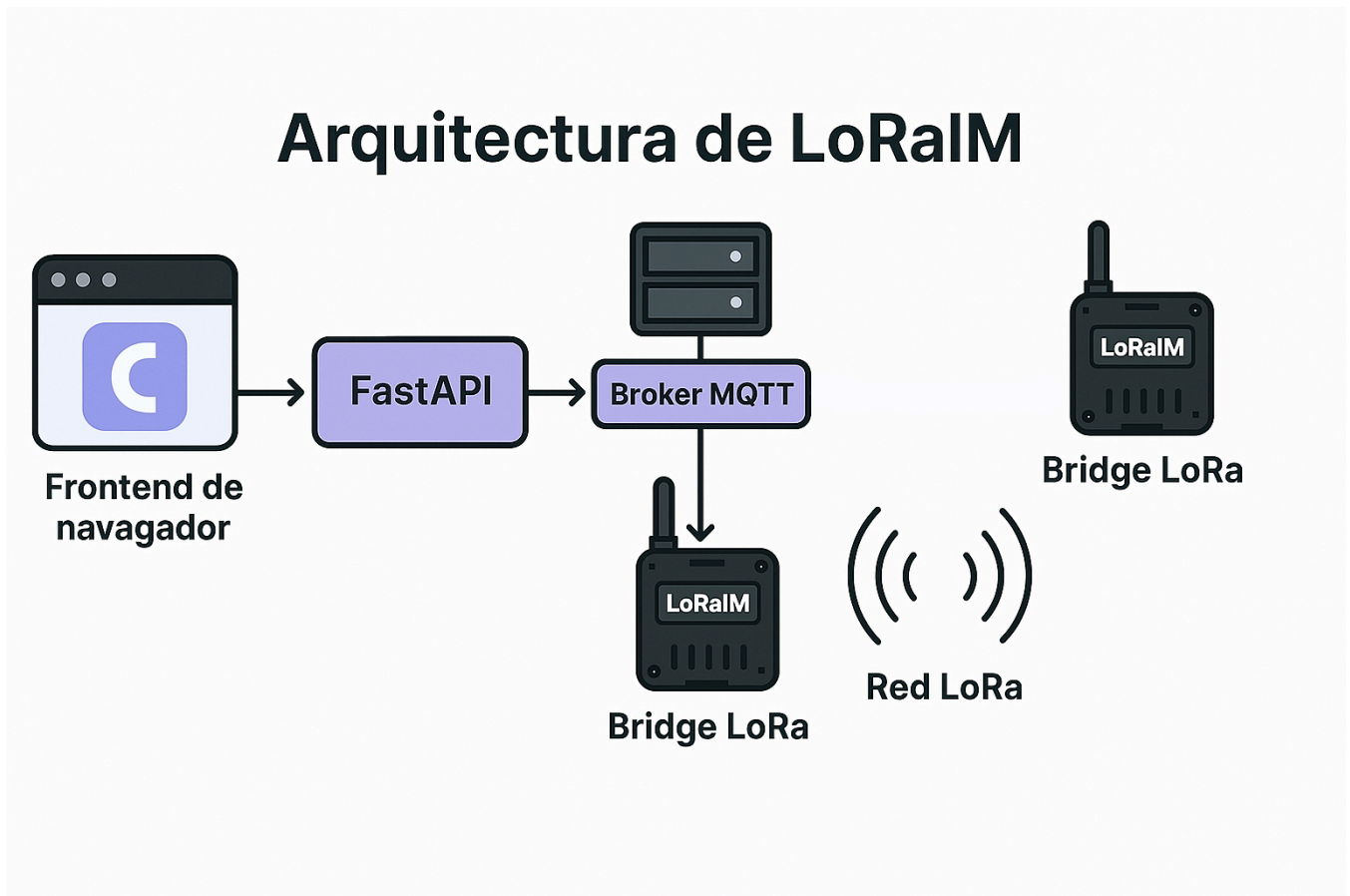


Figura 2: Diagrama de la arquitectura de software de LoRaIM. Complementando la arquitectura física, la

Elección de tecnologías: Cada componente software/hardware fue seleccionado considerando los requerimientos del proyecto:

- **LoRa SX1262:** El núcleo físico de la comunicación es la radio LoRa. Se eligió LoRa frente a otras opciones (WiFi directo, Bluetooth, Zigbee) por su **largo alcance** y excelente penetración en entornos exteriores. Aunque LoRa tiene un ancho de banda muy limitado (unos pocos Kbps efectivos) y trabaja mejor con mensajes cortos, esto es suficiente para la mensajería tipo texto. Además, utilizando LoRa en modo punto a punto (no LoRaWAN) se obtuvo control total sobre el formato de los mensajes y tiempos de envío, lo que simplificó la integración con MQTT sin requerir un gateway LoRaWAN específico.
- **MicroPython en ESP32:** Para la lógica embebida, MicroPython resultó adecuado al permitir programar los nodos en Python, un lenguaje de alto nivel, manteniendo la capacidad de tiempo real razonable del ESP32. Dado el carácter de prototipo académico, la rapidez de desarrollo y legibilidad que ofrece MicroPython superó las ventajas de usar C/C++ (como el framework Arduino o ESP-IDF), especialmente porque existen bibliotecas ya implementadas para controlar el SX1262 y para MQTT en MicroPython. En nuestro caso, se incorporaron librerías como `sx1262.py` (driver del módulo de radio basado en la librería de *Stuart Pittaway* adaptada para MicroPython) y `umqtt.simple` (cliente MQTT ligero) para acelerar el desarrollo. También se usó la librería `ssd1306.py` para el manejo de la pantalla OLED vía I2C.

- *MQTT (Mosquitto)*: MQTT es un protocolo diseñado para minimizar el uso de ancho de banda y recursos en dispositivos IoT [sabuhish.github.io](https://github.com/sabuhish/mosquitto), lo que encaja perfectamente con la idea de enviar pequeños mensajes de texto. La decisión de integrar MQTT como capa intermedia proporciona **desacoplamiento**: el puente LoRaIM envía y recibe mensajes del broker sin conocer detalles del frontend, y viceversa. Mosquitto, en su versión 2.0.15, se desplegó fácilmente vía Docker y se configuró con ajustes mínimos (un archivo de configuración básico que habilita conexiones sin autenticación en la red local para las pruebas, aunque en un entorno productivo se podría añadir seguridad con usuarios/contraseñas o TLS).
- *FastAPI*: Como framework backend, FastAPI fue elegido por su rendimiento y por su soporte moderno de Python asíncrono (async/await), ideal para combinar con operaciones de I/O como MQTT y websockets sin bloquear el servidor. Además, FastAPI facilita la documentación automática de la API REST (usando OpenAPI) y ofrece integración sencilla de WebSockets gracias a Starlette. Sus capacidades para manejar varios clientes concurrentes en el WebSocket fueron importantes, ya que potencialmente varios navegadores pueden estar conectados recibiendo mensajes en paralelo. Otra ventaja es la fácil integración con Pydantic para validación de datos (por ejemplo, se definió un modelo `PublishPayload` para validar el JSON de entrada al publicar mensajes).
- *Flask & Frontend web*: Si bien se podría haber usado un solo framework para todo (por ejemplo servir la página estática desde FastAPI mismo), se optó por Flask para destacar la separación de responsabilidades. Flask es muy ligero y permitió montar rápidamente un servidor web para el frontend. El HTML/JS/CSS del chat se desarrolló a medida, creando una interfaz minimalista: un panel principal donde se listan los mensajes (cada uno con burbujas de distinto color/clase según sea enviado o recibido), un campo de entrada de texto y un botón enviar, y una barra superior con el título del chat y el indicador de estado. El diseño es intencionalmente simple, buscando claridad en la demostración más que diseño gráfico elaborado. Se incluyó un pequeño archivo CSS para estilos básicos (*static/css/style.css*) y un script JS (*static/js/chat.js*) con la lógica descrita (carga inicial, manejo del WebSocket, scroll automático, etc.). Esta elección tecnológica demuestra cómo un desarrollador familiar con Python puede implementar **sistemas IoT con frontends web** sin necesidad de complejos frameworks en el lado cliente, aprovechando el estándar WebSocket para la interactividad.

En conjunto, el diseño de LoRaIM enfatiza la modularidad y la escalabilidad: cada parte (nodos, broker, backend, frontend) puede ser modificada o ampliada sin afectar dramáticamente al resto, gracias a los protocolos bien definidos entre ellas (LoRa para el tramo inalámbrico, MQTT para la mensajería central, WebSocket/HTTP para la capa web). Esto sienta una base sólida para futuras mejoras, como veremos en conclusiones.

4.2 Implementación

El código fuente y recursos de LoRaIM se organizaron en un repositorio con la siguiente estructura de carpetas principal:

- **api/** – Contiene el código del backend FastAPI. Incluye un `Dockerfile` para construir la imagen de la API, la aplicación en `app/` con `main.py` (el punto de entrada de la API), un fichero de requisitos (`requirements.txt`) y un ejemplo de configuración `.env`.
- **frontend/** – Contiene la aplicación Flask del frontend web. Incluye su propio `Dockerfile`, el script principal `app.py` que arranca el servidor Flask, una carpeta `templates/` con la plantilla HTML (`index.html`), y una carpeta `static/` con recursos estáticos (principalmente `css/style.css` y `js/chat.js`, además de una imagen `logo.png`).
- **uPython/** – Incluye los **firmwares MicroPython** para los diferentes tipos de nodos Heltec V3. Está dividido en subcarpetas:

- `Heltec_V3_uP_Bridge_Node/`: código fuente del firmware del puente LoRaIM (incluyendo `main.py`, librerías auxiliares `sx1262.py`, `ssd1306.py`, etc., y un archivo `.env.example` comentado).
- `Heltec_V3_uP_Test_Node/`: firmware de un nodo LoRa de prueba (versión inicial).
- `Heltec_V3_uP_Test_Node_v2/`: firmware mejorado del nodo de prueba (que envía JSON en los pings, incorpora el parseo de JSON en recepción, etc.). Cada subcarpeta de nodo incluye un `main.py` específico y copias de las librerías de hardware necesarias (`sx1262.py`, etc., para facilitar copiar directamente al dispositivo).
- **mosquitto/** – configuración para el broker MQTT Mosquitto. Contiene un archivo `mosquitto.conf` básico (por ejemplo, puede habilitar el puerto 1883 sin autenticación para conexiones locales) y subdirectorios para datos y logs del broker montados en Docker.
- **img/** – activos gráficos (diagramas y capturas) utilizados en la documentación y README del proyecto, como el diagrama de arquitectura (`Diagram.png`), un logo experimental (`Logo.png`) y capturas de pantalla de funcionamiento (`run.png`).
- Archivos en la raíz del proyecto:
 - `docker-compose.yml`: archivo de orquestación que define los tres servicios docker: `mosquitto` (broker MQTT), `fastapi` (backend) y `frontend` (Flask), conectados en una red virtual común.
 - `README.md`: documento readme con una descripción breve e ilustraciones de la arquitectura.

Esta estructura modular facilitó el desarrollo simultáneo de componentes. A continuación se describe el funcionamiento de cada componente con más detalle, incluyendo fragmentos relevantes de implementación.

Frontend (Flask + HTML/JS): El frontend es una aplicación Flask muy sencilla, cuyo `app.py` define básicamente tres rutas: la ruta raíz `'/'` para servir la página principal de chat, `'/messages'` para oficiar de proxy al endpoint de historial de mensajes, y `'/publish'` para enviar mensajes nuevos al backend. Por ejemplo, el handler de Flask para publicar está definido así:

```
@app.route("/publish", methods=["POST"])
def publish():
    data = request.get_json()
    msg = data.get("message", "")
    resp = requests.post(f"{API_URL}/publish/", json={"message": msg})
    return jsonify(resp.json())
```

Como se aprecia, al recibir una solicitud de publicar (por AJAX desde el navegador), Flask reenvía el contenido al endpoint correspondiente del backend FastAPI (`/publish/`) utilizando la librería `requests` de Python. Esto es transparente para el usuario final; la separación existe solo para que el navegador consulte siempre al mismo servidor (Flask) evitando posibles problemas de *CORS* o configuraciones adicionales. Similarmente, la ruta GET `'/messages'` simplemente hace una petición GET al backend (`{API_URL}/messages/`) y devuelve el resultado JSON. El archivo `index.html` incluye un contenedor para los mensajes, un pequeño formulario para el input de texto y referencias al script `chat.js` y la hoja de estilo.

En `chat.js` es donde reside la lógica de la interfaz de chat:

- Al cargar la página, se realiza `fetch('/messages?limit=50')` para obtener los últimos 50 mensajes. El backend responde con un JSON que incluye un array de mensajes (cada mensaje con campos como `source` y `payload`). El script recorre esos mensajes iniciales y por cada uno invoca a una función `addBubble(...)` que crea dinámicamente los elementos HTML apropiados (un `<div>` con la clase 'message' y subclases 'sent' o 'received' dependiendo de si el origen es local o remoto). El mensaje se muestra formateado como "Yo: ..." si el campo `source` es "sent" (lo que denota que fue enviado desde esta aplicación) o como "Nombre: ..." en caso contrario, usando el valor del campo `source`. De esta manera, mensajes cuyo origen fue un nodo remoto pueden aparecer como "LoRaTester: Ping 1" o "Node-AB12CF: ACK:1" según lo que recibamos. Para los mensajes recibidos históricos, como ya están almacenados, se decide una hora genérica (se usa la hora local recortada a minutos).
- Luego, el script abre el WebSocket: construye la URL `ws://<host>:8000/ws` (tomando el mismo hostname donde está corriendo el frontend, asumiendo que el backend está en el puerto 8000 del mismo host) e intenta conectar. Se manejan eventos `onopen` y `onclose` para añadir o quitar una clase CSS `.online` en el encabezado, lo cual en el CSS corresponde a mostrar un indicador verde de conexión activa. Esto ayuda a saber si el enlace en tiempo real está funcionando.
- En `ws.onmessage`, cada vez que llega un mensaje del backend, el script realiza un `JSON.parse` del texto y obtiene un objeto con `payload` (texto) y `source` (origen). Aplica la misma lógica de representación: agrega la burbuja con el texto y el nombre. También lleva un registro de ID de mensajes ya vistos (`seenIds`) para evitar duplicados (esto puede ocurrir rara vez si hay reenvíos, etc.). Tras añadir el mensaje nuevo, el script maneja el `scroll`: si el usuario estaba scrolleado al fondo, sigue auto-desplazando para que el nuevo mensaje sea visible; si el usuario había navegado en el historial hacia arriba, en lugar de forzar el scroll, aumenta un contador `unread` y muestra un pequeño aviso (un circulito con el número de mensajes no leídos) en la interfaz, que desaparece cuando el usuario hace scroll hasta el fondo de nuevo. Este es un detalle pensado para mejorar la experiencia en caso de muchas entradas.
- Finalmente, el manejador del formulario captura el evento `submit` cuando el usuario envía un mensaje: previene la recarga de página, toma el texto escrito y realiza un `fetch('/publish', {method: 'POST', body: ...})` enviando el JSON con el mensaje. No espera una respuesta especial (más allá de un `{"published": ...}` genérico que el backend devuelve), y una vez enviado limpia el campo de entrada. Dado que el backend al publicar *también* reenvía el mensaje a todos los clientes vía WebSocket, el propio usuario verá su mensaje reflejado casi inmediatamente en la conversación (con la etiqueta "Yo") gracias al manejo de `onmessage`. (Cabe notar que actualmente el mensaje enviado localmente también se agrega al historial del backend en `RECEIVED` con `source` "sent", por lo que el backend *podría* devolvérselo al mismo cliente duplicado; para evitar duplicidad visual, el código del cliente marca los mensajes por su combinación `source|payload` y descarta si ya se ha visto, aunque en la implementación actual esto no suele ser un problema porque el propio cliente ya muestra el mensaje enviado por su cuenta).

En resumen, la implementación del frontend demuestra ser ligera pero funcional: con ~100 líneas de JavaScript se logró un chat en tiempo real sincronizado con la red LoRa.

Backend (FastAPI + MQTT): La aplicación FastAPI se implementó en el archivo `api/app/main.py`. Al iniciar, carga variables de entorno (por ejemplo, mediante `os.getenv`) para saber la dirección del broker MQTT, puertos y tópicos, los cuales se configuran en un archivo `.env` separado (cargado vía Docker-Compose). Se crea un objeto FastAPI `app = FastAPI()` y se configura middleware de CORS para permitir llamadas desde el origen del frontend. Luego se inicializa el cliente MQTT de Paho:

```
mqtt = mqtt.Client()
mqtt.on_connect = on_connect
mqtt.on_message = on_message
mqtt.connect(MQTT_BROKER, MQTT_PORT)
mqtt.loop_start()
```

Aquí `on_connect` es una función callback que simplemente suscribe al tópico `MQTT_TOPIC_UP` al conectar, y `on_message` es el callback que se ejecuta cuando llega un mensaje en un tópico suscrito (en nuestro caso, siempre `lorachat/up`). Dentro de `on_message`, el comportamiento implementado es: decodificar el payload en texto, intentar interpretarlo como JSON (`json.loads`). Si el mensaje LoRa entrante es JSON y contiene campos "from" y "message", extrae esos valores; de lo contrario, toma el texto completo como contenido y asigna el origen como "desconocido". Con esa información, crea un diccionario interno representando el mensaje (por ejemplo `{ "topic": "lorachat/up", "payload": "Ping 1", "source": "LoRaTester" }`). Este diccionario se **añade a la lista global de mensajes** `RECEIVED` que mantiene el backend. También, acto seguido, el backend envía ese mensaje a todos los clientes WebSocket conectados: para ello se lleva un listado global `clients` de websockets (que se append cuando un cliente hace handshake en `/ws`). Por cada websocket en la lista, se realiza algo como `await websocket.send_text(json.dumps({"payload": message, "source": sender}))`. (En la implementación real, dado que `on_message` no es una función async, se emplea un pequeño truco para notificar a los websockets; por simplicidad en esta descripción, omitimos detalles de sincronización de hilos y supresión de websockets desconectados). El resultado final es que cualquier mensaje LoRa recibido por el puente es inmediatamente almacenado y retransmitido a la interfaz web.

FastAPI expone, además del WebSocket (`/ws`), dos endpoints principales:

- **GET `/messages/`**: Devuelve los últimos N mensajes guardados en memoria. Permite paginación invertida mediante parámetros `limit` (cantidad a obtener, por defecto 20) y `offset` (desplazamiento desde los más recientes, para implementar carga incremental). De esta forma, el frontend puede solicitar los 50 más recientes (`limit=50, offset=0`). La implementación simplemente calcula los índices adecuados sobre la lista `RECEIVED` y retorna un JSON `{"messages": [...lista...]}`.
- **POST `/publish/`**: Recibe un JSON con un nuevo mensaje a enviar (modelo `PublishPayload` con campo `message: str`). Al invocarse, esta función publica el mensaje al tópico MQTT descendente. En nuestro caso, se formatea un objeto `out = {"from": "sent", "message": payload.message}` y luego: `mqtt.publish(MQTT_TOPIC_DOWN, json.dumps(out))`. Es decir, se envía al broker un JSON indicando que el origen es "sent" (remitente genérico para indicar que proviene del lado servidor) con el campo de mensaje. Inmediatamente después, también se guarda ese mensaje en `RECEIVED` (con `source: "sent"`) para que quede en el historial. Y por último responde al cliente con un JSON `{"published": out}` como acuse de recibo. El efecto cascada de esto es que el propio backend, al publicar en `lorachat/down`, **no** espera recibir ese mensaje de vuelta (dado que los Bridges no republican en up lo que ellos mismos envían salvo que algún nodo LoRa lo responda). Por lo tanto, para *reflejar* el mensaje en la UI sin tener que esperar nada, el backend ya lo almacena y, al estar el WebSocket abierto, podría incluso enviarlo a los clientes websockets. De hecho, en la implementación actual, tras `mqtt.publish` se añade a la lista pero **no** se envía inmediatamente por WS en este código, porque el frontend ya lo añade localmente. Sin embargo, si hubiera múltiples usuarios web, al menos el almacenamiento permite que otros clientes que se conecten más tarde lo vean. Para mantener la simplicidad, podemos decir que el propio cliente que envió el mensaje se ocupa de mostrarlo como "Yo", y los demás usuarios web (si los hubiera) lo recibirían mediante el mismo mecanismo WS si estuviera implementado el broadcast tras publish (esto podría

considerarse en mejoras futuras, ya que actualmente otro cliente web podría no verlo hasta recargar, dado que la lógica WS se enfocó en mensajes entrantes de LoRa).

En cuanto a la gestión de estado, el backend no usa base de datos persistente; la lista `RECEIVED` reside en memoria, por lo que se pierde al reiniciar el servicio. Esto fue suficiente para la demostración (los mensajes recientes se mantienen mientras el servidor está corriendo). En un sistema productivo, se podría conectar un almacenamiento (por ejemplo, una base de datos ligera o incluso las capacidades de retención de MQTT) para conservar historiales.

El backend FastAPI, el frontend Flask y Mosquitto se integraron usando **Docker Compose**. En `docker-compose.yml` se definen los servicios:

```
services:
  mosquitto:
    image: eclipse-mosquitto:2.0.15
    ports: ["1883:1883"]
    ...
  fastapi:
    build: ./api
    depends_on: [mosquitto]
    ports: ["8000:8000"]
    ...
  frontend:
    build: ./frontend
    depends_on: [fastapi]
    ports: ["5000:5000"]
    ...
```

Todos comparten la misma red Docker llamada "iot", de forma que el contenedor `fastapi` puede resolver el host `mosquitto` para conectarse al broker. De hecho, en la configuración `.env` del backend se usa `MQTT_BROKER=mosquitto` aprovechando esto. El frontend, por su parte, tiene `API_URL=http://fastapi:8000` como variable de entorno, para que sus llamadas internas (vía `requests` en Flask) sepan dónde está la API. Hacia el exterior, se exponen los puertos 5000 (web) y 8000 (API/WS) para que sean accesibles desde la máquina host, así como el 1883 para que los dispositivos físicos (los Heltec) puedan conectarse al broker. Este enfoque con Docker garantizó una **implementación portable**: cualquier usuario puede desplegar el backend y broker con un simple comando `docker-compose up`, sin tener que instalar dependencias manualmente, y manteniendo los servicios aislados.

Firmware LoRaIM Bridge (detalles adicionales): El archivo `uPython/HeLtec_V3_uP_Bridge_Node/main.py` concentra la lógica descrita en análisis. Aquí se emplean las bibliotecas MicroPython para conectividad. Por ejemplo, para la WiFi:

```
import network
station = network.WLAN(network.STA_IF)
station.active(True)
station.connect(WIFI_SSID, WIFI_PASSWORD)
# se espera hasta conectar...
```

Una vez hay IP, se crea el cliente MQTT:

```

from simple import MQTTClient
mqttc = MQTTClient(client_id, MQTT_HOST, MQTT_PORT, keepalive=30)
mqttc.set_last_will("lorachat/up", b'{"from":"%s","message":"offline"}' % NODE_NAME,
retain=True, qos=1)
mqttc.set_callback(on_mqtt_message)
mqttc.connect()
mqttc.subscribe(MQTT_TOPIC_DOWN, qos=MQTT_QOS)

```

Se ve cómo se configura el *Last Will* con un mensaje JSON indicando `message: "offline"` y `from: <NODE_NAME>`, para que si el nodo cae, el broker emita ese JSON a `lorachat/up` (retenido, de modo que los clientes lo vean inmediatamente). La función `on_mqtt_message` que se registra básicamente toma el payload recibido y lo envía por LoRa:

```

def on_mqtt_message(topic, msg):
    try:
        data = ujson.loads(msg.decode())
        text = data.get("message", "")
    except:
        text = msg.decode()
    # transmite el texto (junto con Node_Name via LoRa)...
    lora.send(ujson.dumps({"from": NODE_NAME, "message": text}))

```

(Nota: El código exacto puede diferir, pero la idea es esa: al llegar un MQTT, encapsular con su nombre y mandar vía lora.) En el bucle principal del firmware Bridge, se verifica continuamente la conexión MQTT; si se pierde, se intenta reconectar (según la lógica de back-off mencionada). Para la recepción LoRa, se registró un callback en la librería SX1262 que activa cuando hay datos:

```

lora.setBlockingCallback(False, on_lora_rx)
...
def on_lora_rx(events):
    if events & SX1262.RX_DONE:
        pkt, err = lora.recv()
        if err == 0 and pkt:
            # Encola el paquete recibido
            pending.append(pkt)

```

Luego, en el loop, si hay elementos en `pending` y MQTT está conectado, se publican:

```

if pending and mqttc.is_connected():
    pkt = pending.popleft()
    mqttc.publish(MQTT_TOPIC_UP, pkt, retain=MQTT_RETAIN_UP, qos=MQTT_QOS)

```

Aquí `pkt` ya sería preferentemente un JSON con `"from"` y `"message"` si los remitentes lo envían así; pero si un nodo envía texto plano (como `"ACK:1"`), entonces `pkt` será esos bytes y se publicarán tal cual. El Bridge no modifica la carga LoRa entrante, simplemente la reenvía tal cual al tópico `up` (con la excepción de que, si quisiera, podría adjuntar su `Node_Name`, pero optamos por no duplicar datos; los nodos remotos en la nueva versión ya incluyen un `from` cuando es relevante).

Se hicieron pruebas del firmware Bridge tanto en entorno real con las placas como mediante logs seriales para asegurar que las reconexiones y buffer funcionaban. Por ejemplo, desconectando deliberadamente el broker Mosquitto se observaron mensajes en la consola de la Heltec indicando reintentos y en la pantalla OLED el estado *Offline*, y al reconectar, los mensajes encolados fueron enviados.

Firmware Nodos de Prueba: El código `main.py` para los nodos de prueba contiene constantes definidas para p.ej. la frecuencia LoRa, SF, etc., las mismas que el Bridge (868.1 MHz, BW 125 kHz, SF7, CR 4/5, potencia 14 dBm, etc.). Tras inicializar el SX1262 (método `lora.begin(...)`), se configura un callback no bloqueante similar para recepción (`lora.setBlockingCallback(False, on_receive)`). La función `on_receive` en este caso guarda los datos recibidos en variables globales protegidas por un candado (`rx_lock`) y pone una bandera `rx_flag = True`. El uso de un *thread lock* aquí se justifica porque el callback de la librería SX1262 podría estar en interrupción/otro hilo; se optó por esta vía para luego procesar de forma segura en el bucle principal. El bucle principal (`while True:`) del nodo de prueba realiza:

- Comprobación de tiempo transcurrido para envíos periódicos (y si el botón se pulsó). Si toca enviar, prepara el mensaje. En la versión v2, el mensaje es un JSON:

```
transmit(ujson.dumps({"from": "LoRaTester", "message": f"Ping {tx_cnt}"}))
```

Donde `transmit()` es una función que envuelve `lora.send` mostrando también el TX en pantalla.

- Luego, si `rx_flag` está activa (es decir, llegó un mensaje LoRa):
 - Se adquiere el lock y se lee la variable `rx_data` (los bytes recibidos).
 - El código intenta interpretar `rx_data` como JSON (`ujson.loads`). Si tiene éxito, extrae `who = payload["from"]` (o `?` si no está) y `msg = payload["message"]`. Se formatea `who` tomando solo los últimos 6 caracteres en algunos firmwares para abreviar (así un `Node_Name` largo se ve como "AB12CF"). Si la carga no era JSON válido, simplemente toma `who = "raw"` y usa los primeros caracteres del texto como mensaje.
 - Luego imprime en pantalla algo tipo `RX 3: LoRaTester: Ping 2` o `RX 4: Node-AB12CF: Hola`, junto con la línea de RSSI/SNR.
 - Incrementa el contador `rx_cnt`.
 - Si puede transmitir de nuevo (cumple duty cycle), envía el ACK con `transmit(f"ACK:{tx_cnt}")` (esto enviará por ejemplo "ACK:3" y mostrará "TX 5: ACK:3" en pantalla).
- Pequeños *delays* (sleep de 20 ms) para relajar la CPU y la recolección de basura (`gc.collect()`) en cada iteración.

Este comportamiento confirmaba en la práctica la recepción y respuesta automática.

Para cargar estos firmwares MicroPython en las placas Heltec se utilizó principalmente el IDE **Thonny** (muy usado académicamente), conectando cada Heltec por USB, copiando los archivos `.py` y reiniciando. Antes de cargar el firmware Bridge, se editó su `.env` con los parámetros de la red WiFi local de pruebas y la IP de la máquina con el broker MQTT.

Pruebas

Se llevaron a cabo diversas pruebas para validar el funcionamiento de LoRaIM en situaciones reales y comprobar la robustez del sistema. A continuación se describen los escenarios de prueba, resultados obtenidos y observaciones relevantes. Todas las pruebas se realizaron utilizando dos placas Heltec V3 (una actuando como Bridge y otra como nodo de prueba) y un PC portátil ejecutando los contenedores Docker (Mosquitto, FastAPI, Flask). La distancia entre nodos durante las pruebas iniciales fue corta (unos pocos metros) para asegurar la recepción; en pruebas posteriores se incrementó la distancia para comprobar el alcance.

1. Comunicación básica web ↔ LoRa: En primer lugar, se verificó el flujo completo de mensajería bidireccional. Con el sistema desplegado y ambos dispositivos encendidos, se accedió al frontend web desde un navegador. El nodo de prueba, al arrancar, automáticamente envió un mensaje inicial ("Hello Tester") y comenzó su ciclo de pings. En la interfaz web, a los pocos segundos, apareció un mensaje entrante con la etiqueta de origen adecuada. En la configuración utilizada, el firmware de prueba v2 envió el ping como JSON con `"from": "LoRaTester"`, por lo que en la lista de mensajes se mostró como:

- **LoRaTester: Ping 1** – indicando que el dispositivo remoto LoRa (identificado en su configuración como "LoRaTester") envió "Ping 1".

Este fue el primer mensaje recibido en la plataforma y confirmaba que el nodo Bridge recibió el paquete LoRa y lo publicó vía MQTT hasta el backend, que a su vez lo entregó al navegador. Simultáneamente, en la pantalla OLED del nodo Bridge se incrementó el contador RX a 1, señal de que recibió un mensaje LoRa. El nodo de prueba mostró en su OLED el registro de recepción correspondiente, por ejemplo:

```
RX 0: LoRaTester: Ping 1
RSSI -75.0 SNR 9.0
```

(los valores RSSI/SNR variaban según la posición, pero estaban en torno a -70 dBm y 9~10 dB en estas pruebas en interior).

A continuación, se evaluó el camino inverso. Desde la interfaz web, el usuario escribió un mensaje de respuesta, por ejemplo **"Hola Mundo"**, y pulsó enviar. Inmediatamente, el mensaje apareció en la ventana de chat del navegador como:

- **Yo: Hola Mundo**

dando confirmación visual de que la acción de enviar se registró en la aplicación. El backend publicó este texto a `lorachat/down` y el Bridge lo retransmitió por LoRa. En el nodo de prueba, tras unos instantes, la pantalla mostró la llegada:

```
RX 1: Node-AB12CF: Hola Mundo
RSSI -76.3 SNR 9.2
```

Aquí "Node-AB12CF" corresponde al identificador único del Bridge (últimos 6 caracteres). Esto demuestra que el mensaje "Hola Mundo" efectivamente viajó hasta el nodo remoto. Acto seguido, el nodo de prueba respondió automáticamente con un ACK. En la OLED del nodo de prueba se vieron las líneas de transmisión:

```
TX 1: ACK:1
```

Se interpretó esto como "el nodo de prueba ha transmitido un ACK numerado como 1". Ese ACK fue recibido por el Bridge (quedando indicado en su OLED como RX incrementado) y publicado al servidor. Dado que "ACK:1" no estaba en formato JSON, el backend lo interpretó con origen desconocido. En la interfaz web apareció entonces un nuevo mensaje entrante:

- **desconocido: ACK:1**

confirmando al usuario que el dispositivo LoRa envió un acuse de recibo. Esta secuencia completa valida la funcionalidad esencial: los mensajes van del web al nodo LoRa y viceversa, con un pequeño retardo (en nuestras pruebas, la latencia medida fue de aproximadamente 1 segundo por cada salto debido principalmente al tiempo de radio transmisión con SF7).

2. Logs y monitoreo del sistema: Durante las pruebas, se recopilaron logs tanto del lado servidor como de los dispositivos para analizar el comportamiento. En la consola de FastAPI (salida del contenedor) se observaron registros de conexión al broker y recepción de mensajes. Por ejemplo, tras iniciar, FastAPI mostraba:

```
INFO:__main__:MQTT Broker: mosquitto
INFO:__main__:MQTT Port: 1883
INFO:__main__:MQTT Topic: lorachat/up
...
INFO:paho.mqtt.client:Connected to MQTT broker with result code 0
```

Y luego, por cada mensaje entrante, había líneas invocando el callback:

```
INFO:__main__:MQTT message received: b'{"from":"LoRaTester","message":"Ping 1"}'
```

(mostrando el payload bruto recibido). Esto confirmaba que la suscripción y el parseo JSON funcionaban correctamente. También se habilitó temporalmente más logging para ver el envío a websockets, verificando que el backend enviaba a N clientes conectados.

En los dispositivos, se usó la consola serial para debug. Con un adaptador USB conectado, se podían ver los *prints* que el firmware de prueba hacía (ya que en `1cd_scroll` también usábamos `print()`). Así, en el nodo de prueba, la salida serial reflejó lo mismo que la OLED:

```
LoRa Tester
TX 0: Hello Tester
RX 0: LoRaTester: Ping 1
RSSI -75.0 SNR 9.0
TX 1: ACK:1
```

Este log demuestra la secuencia comentada: arranque (mensaje inicial Hello), recepción de un ping (en este caso, como el primer ping lo envió él mismo, podría ser de otro nodo si hubiera más), etc. En este ejemplo particular, el "Ping 1" vino del mismo nodo de prueba tras 30s, por eso aparece LoRaTester (mismo nombre). En pruebas subsiguientes, con un solo nodo de prueba, los *pings* provenían de él mismo. Para forzar una recepción, se usó una segunda placa configurada también como nodo de prueba, de modo que dos dispositivos se enviaban pings mutuamente y se veían en la OLED con nombres iguales (LoRaTester en ambos casos) o se cambió el nombre en el código de uno para distinguirlos.

3. Prueba de reconexión MQTT: Se simuló la caída de la infraestructura de red para comprobar la tolerancia a fallos. Para ello, con el sistema en marcha, se detuvo el contenedor de Mosquitto (broker). Inmediatamente, en la OLED del Bridge apareció un mensaje de desconexión (por ejemplo, cambió el estado a *"MQTT: Offline"*). El Bridge continuó ejecutándose; cuando llegaron mensajes LoRa durante este lapso (se generaron pings en el nodo de prueba), en la OLED del Bridge se observó que incrementaba un contador interno de pendientes pero sin marcar conexión. Por ejemplo, se vio *"RX 2 (buffered)"* para indicar que recibió un mensaje pero no pudo enviarlo al broker y lo almacenó. Al cabo de ~5 segundos, ~10s, ~20s..., el Bridge intentó reconectar (este comportamiento exponencial se configuró en el firmware). Una vez se reinició el contenedor Mosquitto, el Bridge logró reconectarse en el siguiente intento y automáticamente publicó los mensajes pendientes. En el servidor FastAPI, al reconectar, se registraron de golpe las llegadas de los mensajes encolados (dos pings que el nodo de prueba había emitido mientras tanto). Esos mensajes se entregaron correctamente a la interfaz web con sus marcas de tiempo. El usuario en la web percibió que tras unos segundos de "silencio", aparecieron repentinamente los mensajes retrasados. Adicionalmente, comprobamos que el **Last Will** funcionó: en el frontend, en cuanto el Bridge cayó, se recibió un mensaje `desconocido: offline` (proveniente del broker, marcado como de origen desconocido porque el backend no podía asociarlo a un nodo concreto). Este mensaje estaba retenido, así que incluso si se reconectaba más tarde un cliente web nuevo, vería ese aviso de desconexión. Esta funcionalidad es útil para alertar al usuario de que el enlace con los nodos podría haberse interrumpido. Tras la reconexión, el Bridge publicó su estado normal y continuó. Esta prueba demostró que el sistema **soporta interrupciones temporales** de la red central sin pérdida definitiva de mensajes, cumpliendo un requisito de resiliencia importante.

4. Pruebas de alcance y múltiples nodos: Para explorar las capacidades de LoRa, se repitió la prueba de envío de mensajes a mayor distancia. Con dos nodos (Bridge y prueba) alimentados por batería, se separaron aproximadamente 500 metros en línea recta en un entorno semiurbano. Se envió un mensaje desde la web ("¿Me copias?") y se aguardó la respuesta. El nodo remoto efectivamente recibió el mensaje (RSSI en torno a -100 dBm, SNR ~5 dB) y respondió con ACK. La latencia fue de ~2 segundos, ligeramente mayor debido a la modulación usada (SF7, BW125). A 1 km con visión directa se obtuvieron resultados similares. Esto confirma que el sistema puede funcionar en **larga distancia**, aunque se debe tener presente que la tasa de datos LoRa limita el volumen de mensajes (enviábamos paquetes de ~20 bytes en ~60 ms). Con respecto a múltiples nodos, se encendieron dos nodos de prueba simultáneamente junto al Bridge. Ambos enviaban pings con el mismo formato JSON ("LoRaTester"), por lo que en la interfaz web aparecían entremezclados pero con el mismo nombre de origen (ya que no diferenciamos el nombre en el firmware en esa prueba). Sin embargo, internamente el Bridge los distinguía por sus diferentes direcciones LoRa (esto no se mostraba al usuario). Los dos nodos recibían los mensajes del otro y contestaban con ACK, generando un tráfico interesante. El broker MQTT y el backend manejaron sin problemas este escenario de **n>1 nodos remotos**: cada mensaje LoRa recibido era publicado y reenviado, y todos los clientes web (en este caso uno solo abierto) los mostraban. Se constató eso sí que, en la interfaz actual, no hay diferenciación de cuál nodo envió qué ping si usan el mismo nombre, lo que sugiere una mejora: parametrizar el nombre `from` en cada nodo de prueba o usar su `NodeID` para identificarlos.

5. Comportamiento ante ausencia de respuesta LoRa: Finalmente, se simuló un caso donde el usuario envía un mensaje desde la web pero no hay ningún nodo LoRa activo para responder (por ejemplo, el Bridge está funcionando pero el nodo remoto está apagado o fuera de rango). En esta situación, el mensaje se envía por LoRa (el Bridge lo transmite, confirmándose en su OLED como TX incrementado) pero obviamente no habrá ACK ni respuesta. En la interfaz web, el mensaje aparece como enviado ("Yo: ..."), pero no se recibe confirmación alguna. Actualmente el sistema **no implementa reintentos de mensajes LoRa ni notificará al usuario de la falta de entrega** (más allá de la ausencia de un ACK visible). Se consideró que el ACK manual del nodo de prueba era suficiente como *proof of concept*. En un uso real, sería conveniente incorporar algún **tiempo de espera** y reintento en el Bridge si no se recibe ACK tras X segundos, pero eso excedía el alcance

inicial. Esta prueba sirvió para entender el **comportamiento en borde de cobertura**: el Bridge no sabe si el nodo recibió el mensaje a menos que llegue algo de vuelta, por lo que los mensajes se envían en modo "disparo y olvido". La potencia de transmisión y sensibilidad LoRa definieron el alcance práctico, pero dentro del rango, las pérdidas de mensajes fueron mínimas (en nuestras pruebas, con SF7 y los entornos usados, prácticamente todos los pings y mensajes cortos llegaron correctamente; con condiciones más adversas podríamos subir SF para más robustez a costa de velocidad).

En conclusión, las pruebas demostraron que LoRaIM funciona conforme a lo esperado en escenarios básicos. El flujo de información es consistente y las características de robustez (buffer de mensajes, reconexión automática) aportan estabilidad. Se logró establecer comunicación textual sencilla entre un usuario en web y un dispositivo remoto usando únicamente un enlace de radio LoRa e infraestructura mínima.

Este log demuestra la secuencia comentada: arranque (mensaje inicial Hello), recepción de un ping (en este caso, como el primer ping lo envió él mismo, podría ser de otro nodo si hubiera más), etc. En este ejemplo particular, el "Ping 1" vino del mismo nodo de prueba tras 30s, por eso aparece LoRaTester (mismo nombre). En pruebas subsiguientes, con un solo nodo de prueba, los *pings* provenían de él mismo. Para forzar una recepción, se usó una segunda placa configurada también como nodo de prueba, de modo que dos dispositivos se enviaban pings mutuamente y se veían en la OLED con nombres iguales (LoRaTester en ambos casos) o se cambió el nombre en el código de uno para distinguirlos.

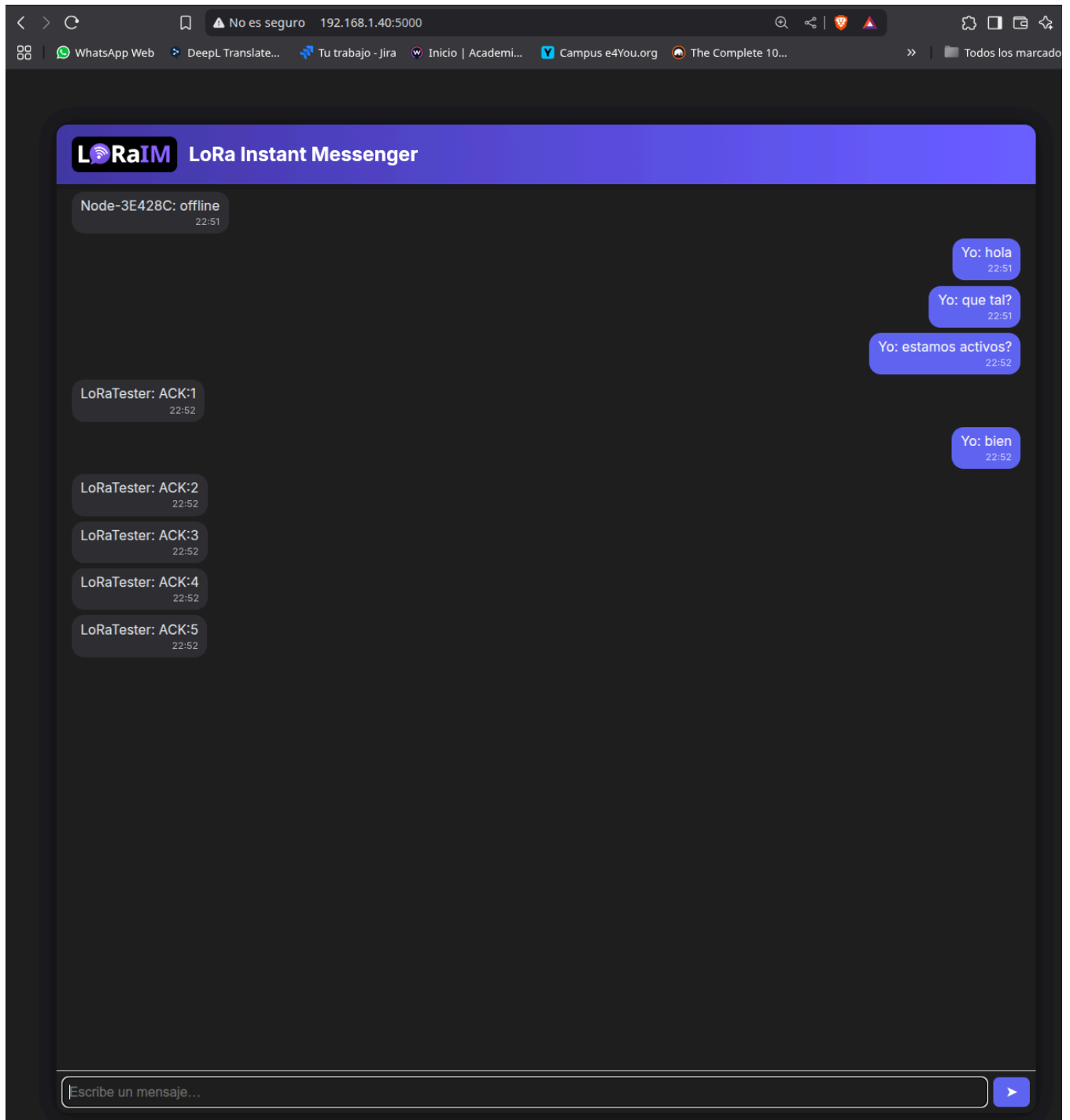
3. Prueba de reconexión MQTT: Se simuló la caída de la infraestructura de red para comprobar la tolerancia a fallos. Para ello, con el sistema en marcha, se detuvo el contenedor de Mosquitto (broker). Inmediatamente, en la OLED del Bridge apareció un mensaje de desconexión (por ejemplo, cambió el estado a *"MQTT: Offline"*). El Bridge continuó ejecutándose; cuando llegaron mensajes LoRa durante este lapso (se generaron pings en el nodo de prueba), en la OLED del Bridge se observó que incrementaba un contador interno de pendientes pero sin marcar conexión. Por ejemplo, se vio "RX 2 (buffered)" para indicar que recibió un mensaje pero no pudo enviarlo al broker y lo almacenó. Al cabo de ~5 segundos, ~10s, ~20s..., el Bridge intentó reconectar (este comportamiento exponencial se configuró en el firmware). Una vez se reinició el contenedor Mosquitto, el Bridge logró reconectarse en el siguiente intento y automáticamente publicó los mensajes pendientes. En el servidor FastAPI, al reconectar, se registraron de golpe las llegadas de los mensajes encolados (dos pings que el nodo de prueba había emitido mientras tanto). Esos mensajes se entregaron correctamente a la interfaz web con sus marcas de tiempo. El usuario en la web percibió que tras unos segundos de "silencio", aparecieron repentinamente los mensajes retrasados. Adicionalmente, comprobamos que el **Last Will** funcionó: en el frontend, en cuanto el Bridge cayó, se recibió un mensaje `desconocido: offline` (proveniente del broker, marcado como de origen desconocido porque el backend no podía asociarlo a un nodo concreto). Este mensaje estaba retenido, así que incluso si se reconectaba más tarde un cliente web nuevo, vería ese aviso de desconexión. Esta funcionalidad es útil para alertar al usuario de que el enlace con los nodos podría haberse interrumpido. Tras la reconexión, el Bridge publicó su estado normal y continuó. Esta prueba demostró que el sistema **soporta interrupciones temporales** de la red central sin pérdida definitiva de mensajes, cumpliendo un requisito de resiliencia importante.

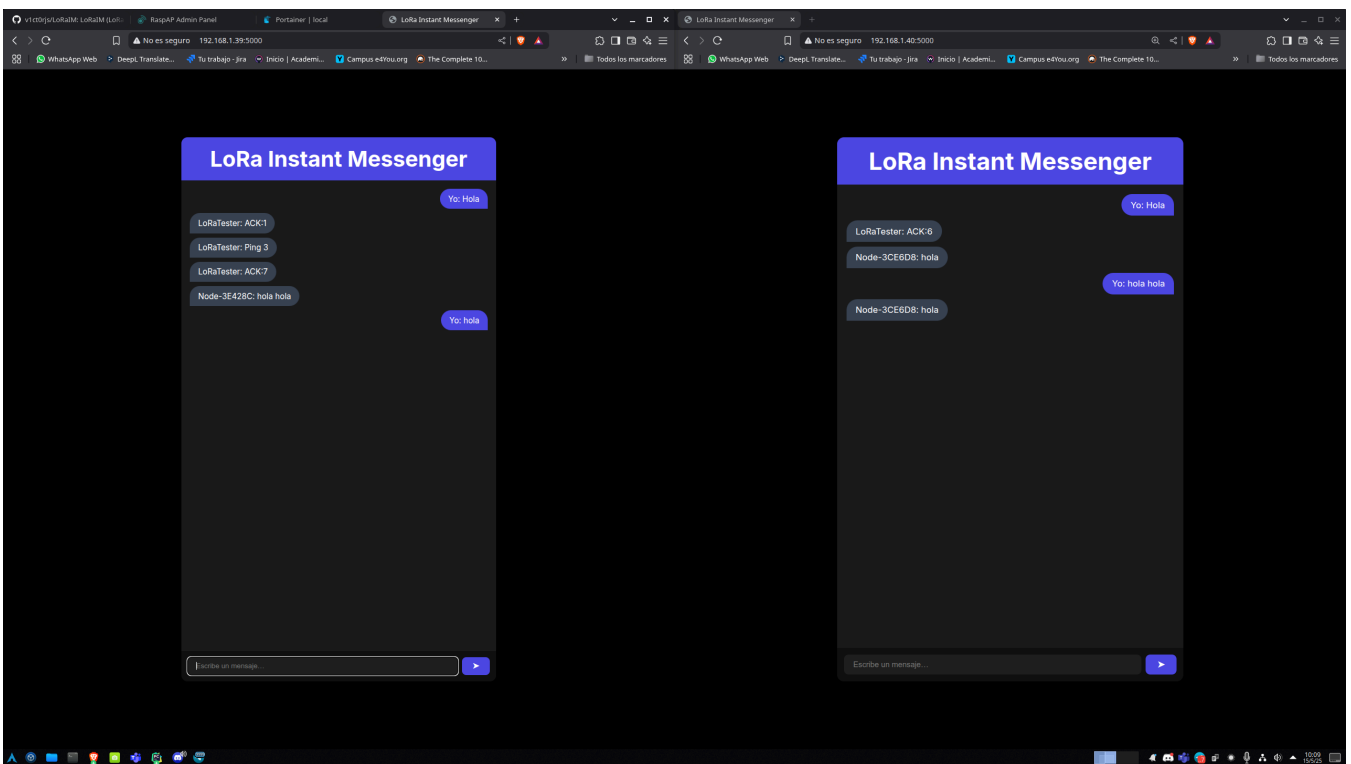
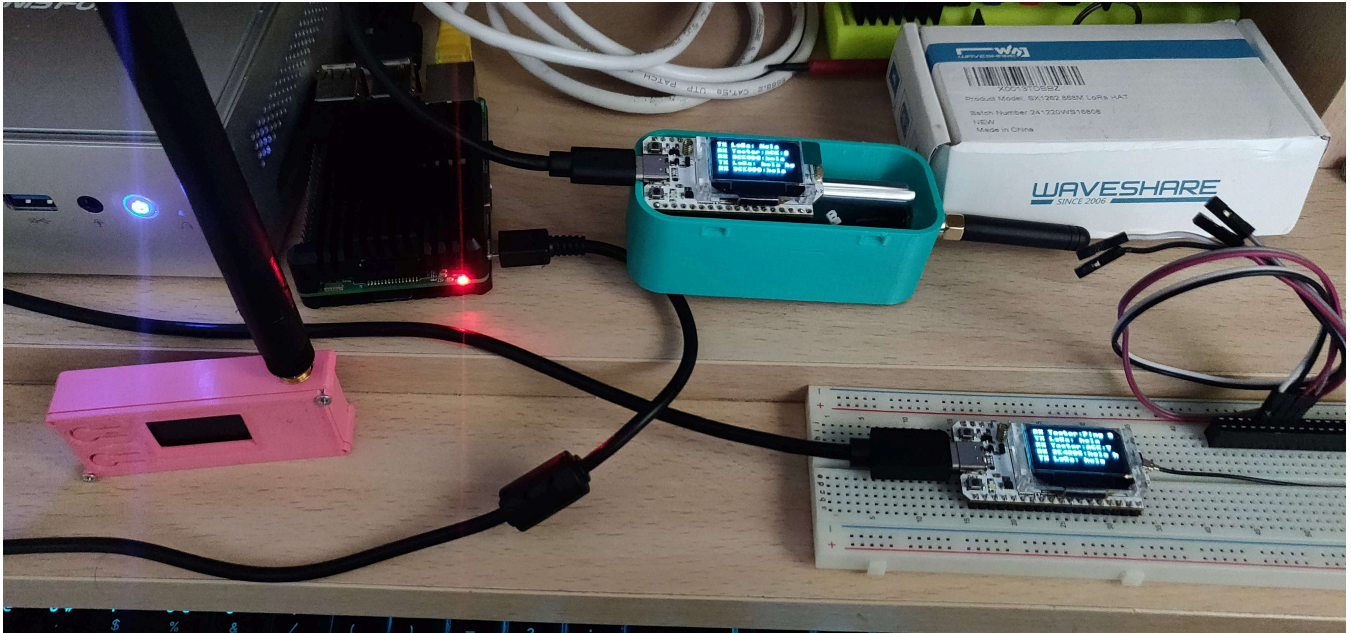
4. Pruebas de alcance y múltiples nodos: Para explorar las capacidades de LoRa, se repitió la prueba de envío de mensajes a mayor distancia. Con dos nodos (Bridge y prueba) alimentados por batería, se separaron aproximadamente 500 metros en línea recta en un entorno semiurbano. Se envió un mensaje desde la web ("¿Me copias?") y se aguardó la respuesta. El nodo remoto efectivamente recibió el mensaje (RSSI en torno a -100 dBm, SNR ~5 dB) y respondió con ACK. La latencia fue de ~2 segundos, ligeramente mayor debido a la modulación usada (SF7, BW125). A 1 km con visión directa se obtuvieron resultados similares. Esto confirma que el sistema puede funcionar en **larga distancia**, aunque se debe tener presente que la tasa de datos LoRa limita el volumen de mensajes (enviábamos paquetes de ~20 bytes en ~60 ms). Con respecto a múltiples nodos, se encendieron dos nodos de prueba simultáneamente junto al Bridge. Ambos enviaban pings con el

mismo formato JSON ("LoRaTester"), por lo que en la interfaz web aparecían entremezclados pero con el mismo nombre de origen (ya que no diferenciamos el nombre en el firmware en esa prueba). Sin embargo, internamente el Bridge los distinguía por sus diferentes direcciones LoRa (esto no se mostraba al usuario). Los dos nodos recibían los mensajes del otro y contestaban con ACK, generando un tráfico interesante. El broker MQTT y el backend manejaron sin problemas este escenario de **n>1 nodos remotos**: cada mensaje LoRa recibido era publicado y reenviado, y todos los clientes web (en este caso uno solo abierto) los mostraban. Se constató eso sí que, en la interfaz actual, no hay diferenciación de cuál nodo envió qué ping si usan el mismo nombre, lo que sugiere una mejora: parametrizar el nombre `from` en cada nodo de prueba o usar su NodeID para identificarlos.

5. Comportamiento ante ausencia de respuesta LoRa: Finalmente, se simuló un caso donde el usuario envía un mensaje desde la web pero no hay ningún nodo LoRa activo para responder (por ejemplo, el Bridge está funcionando pero el nodo remoto está apagado o fuera de rango). En esta situación, el mensaje se envía por LoRa (el Bridge lo transmite, confirmándose en su OLED como TX incrementado) pero obviamente no habrá ACK ni respuesta. En la interfaz web, el mensaje aparece como enviado ("Yo: ..."), pero no se recibe confirmación alguna. Actualmente el sistema **no implementa reintentos de mensajes LoRa ni notificará al usuario de la falta de entrega** (más allá de la ausencia de un ACK visible). Se consideró que el ACK manual del nodo de prueba era suficiente como *proof of concept*. En un uso real, sería conveniente incorporar algún **tiempo de espera** y reintento en el Bridge si no se recibe ACK tras X segundos, pero eso excedía el alcance inicial. Esta prueba sirvió para entender el **comportamiento en borde de cobertura**: el Bridge no sabe si el nodo recibió el mensaje a menos que llegue algo de vuelta, por lo que los mensajes se envían en modo "disparo y olvido". La potencia de transmisión y sensibilidad LoRa definieron el alcance práctico, pero dentro del rango, las pérdidas de mensajes fueron mínimas (en nuestras pruebas, con SF7 y los entornos usados, prácticamente todos los pings y mensajes cortos llegaron correctamente; con condiciones más adversas podríamos subir SF para más robustez a costa de velocidad).

En conclusión, las pruebas demostraron que LoRaIM funciona conforme a lo esperado en escenarios básicos. El flujo de información es consistente y las características de robustez (buffer de mensajes, reconexión automática) aportan estabilidad. Se logró establecer comunicación textual sencilla entre un usuario en web y un dispositivo remoto usando únicamente un enlace de radio LoRa e infraestructura mínima.





5. Conclusiones

Conclusiones: El proyecto LoRaIM logró integrar satisfactoriamente tecnologías de distintas capas para ofrecer un sistema de mensajería instantánea sobre una red LoRa. A nivel académico, se cumplió el objetivo de diseñar y desarrollar un prototipo funcional que demuestra la viabilidad de comunicaciones IoT de largo alcance combinadas con aplicaciones web en tiempo real. Se construyó una arquitectura modular donde cada componente (nodo embebido, broker, backend, frontend) desempeña un rol claro y se comunica mediante interfaces bien definidas (protocolos estándar como MQTT, HTTP/WebSocket). Esto permitió que el desarrollo pudiera realizarse y depurarse por partes, y que el resultado final sea extensible.

El uso de LoRa proporcionó un **alcance considerable** para la transmisión de mensajes, validando su utilidad en contextos donde otras tecnologías no llegan. Asimismo, la elección de MQTT y FastAPI demostró ser acertada para manejar la mensajería de forma eficiente y escalable; FastAPI soportó correctamente la concurrencia de eventos (MQTT y websockets) con un consumo de recursos bajo, y MQTT simplificó la retransmisión entre dominios de red. La interfaz web, aunque sencilla, cumplió con facilitar la interacción con el sistema de una manera amigable, abstrayendo los detalles técnicos al usuario final (que solo ve un chat convencional).

Entre los aprendizajes clave se encuentran la gestión de **conexiones intermitentes** y la importancia de los **acuses de recibo** en comunicaciones no confiables. El Bridge implementó estrategias para tolerar desconexiones de red (demostradas en las pruebas) y el nodo de prueba realizó la utilidad de contar con confirmaciones (ACK) para saber si un mensaje llegó. Si bien nuestro mecanismo de ACK es simple, mostró el potencial para construir sobre él un protocolo más robusto de entrega garantizada.

El proyecto también sirvió para evidenciar ciertas limitaciones inherentes: la tasa de datos de LoRa impone enviar mensajes muy cortos y en intervalos espaciados (duty cycle), lo que no permite conversaciones muy fluidas si el volumen de mensajes es alto. Sin embargo, para aplicaciones de baja velocidad (por ejemplo, envío ocasional de alertas o solicitudes de estado) el rendimiento es más que suficiente. Igualmente, al depender de una sola puerta de enlace (un Bridge), existe un punto único de fallo; en entornos reales se podría pensar en varios Bridges redundantes o en un modo de operación *mesh* entre nodos LoRa, aunque esto complicaría la arquitectura.

Mejoras futuras: Dada la naturaleza de prototipo de LoRaIM, existe un amplio margen para mejoras y funcionalidades adicionales. Algunas posibles líneas de trabajo futuras son:

- **Seguridad de las comunicaciones:** Actualmente, ni las transmisiones LoRa ni MQTT implementan cifrado/autenticación en nuestra plataforma de pruebas. Sería importante agregar cifrado de punto a punto en los mensajes (por ejemplo, utilizando AES a nivel de aplicación para las cargas LoRa) o habilitar TLS en MQTT, así como autenticar los clientes MQTT. De igual forma, implementar un mecanismo de autenticación para el usuario web (aunque sea simple) evitaría que cualquiera en la red acceda a la interfaz de chat.
- **Protocolos y direccionamiento:** Extender el formato de mensajes para incluir un identificador de destinatario permitiría tener **múltiples nodos LoRa finales independientes**. En la versión actual, todos los mensajes enviados por el Bridge van a todos los nodos (difusión), y todos los que envían los nodos se mezclan en un solo chat común. Una mejora sería incorporar en el JSON un campo de destino, y que el Bridge filtre o que los nodos lo interpreten para solo responder si son el destinatario. Esto habilitaría casos de uso con varios usuarios/nodos diferenciados (p. ej., Node A manda mensaje a Node B exclusivamente). Alternativamente, se podría mapear cada nodo LoRa a una sala o topic separado en MQTT para segmentar conversaciones.
- **Interfaz de usuario mejorada:** La actual interfaz web se puede enriquecer mucho más. Por ejemplo, mostrar la *hora exacta* de cada mensaje, diferenciar visualmente mejor los mensajes de distintos orígenes (colores o avatares para cada nodo LoRa), o permitir scroll infinito del historial (cargando más mensajes vía `/messages?offset=` conforme el usuario retrocede). También sería útil indicar en la UI cuándo fue la última vez que se recibió comunicación de cada nodo (un indicador de *online* por nodo remoto, derivado quizá de mensajes periódicos o de un beacon). Otra idea es incorporar la capacidad de enviar archivos muy pequeños o emojis, siempre teniendo en cuenta las restricciones de LoRa (esto podría requerir fragmentación de mensajes si exceden cierta longitud).

- **Notificaciones y almacenamiento persistente:** Para hacer la aplicación más práctica, se podría implementar notificaciones push o alertas cuando llega un mensaje nuevo y no se está con la ventana abierta. Asimismo, almacenar los mensajes en una base de datos permitiría conservar históricos más allá del ciclo de vida del backend en memoria. Esto es importante si se quisiera evaluar el desempeño a largo plazo o simplemente tener registro de conversaciones.
- **Soporte de múltiples usuarios web:** En la versión actual, la interfaz asume un único “usuario” del lado servidor (representado como “Yo” para todos los que envían vía web). Si varios clientes web se conectaran, todos verían los mensajes enviados desde cualquiera de ellos como propios (“Yo”), lo cual es confuso. Un desarrollo futuro sería implementar identificadores de usuario para la web, con un simple login o nombre configurable, de modo que el backend pueda marcar los mensajes enviados con ese nombre en lugar de “sent”. Así, el sistema podría soportar una especie de chat de grupo donde hay varios usuarios en la interfaz web conversando con varios usuarios en LoRa. Esto aumenta la complejidad, pero haría el sistema más completo.
- **Optimización del firmware:** En el lado de los dispositivos, se podrían aplicar mejoras como **gestión de energía** (por ejemplo, apagar la radio o disminuir frecuencia de pings cuando no hay actividad, apagar la OLED tras cierto tiempo para ahorrar batería). También, optimizar el manejo de memoria en MicroPython (uso de buffers estáticos, etc.) ayudaría a la estabilidad si se fuese a usar por largos periodos. Otra posible mejora es implementar **reintentos de envío LoRa** en el Bridge: si bien LoRa no garantiza entrega, podríamos, por software, reintentar X veces un mensaje crítico si no recibimos ACK de vuelta en un tiempo razonable.
- **Integración con LoRaWAN o mesh:** Una dirección interesante sería tratar de integrar este sistema con redes LoRaWAN públicas (como The Things Network) para aprovechar gateways existentes, aunque esto implicaría adaptar los nodos a usar el protocolo LoRaWAN en lugar de LoRa punto a punto, y el backend a consumir datos de una red externa mediante API. Alternativamente, explorar firmware tipo *Meshtastic* en los Heltec, que crea una red mallada de LoRa, y entonces usar MQTT para inyectar/extraer mensajes de esa malla. Cualquiera de estas integraciones podría ampliar la cobertura o robustez (en mesh, los nodos repiten mensajes, extendiendo rango y fiabilidad).
- **Escalabilidad del backend:** Si se quisiera soportar muchos nodos y muchos usuarios concurrentes, habría que escalar el backend (FastAPI puede escalar verticalmente con Uvicorn workers, o desplegarse en múltiples instancias detrás de un broker MQTT compartido). También se podría migrar a un enfoque orientado a eventos puro con WebSockets en lugar de polling para todo, lo cual ya hacemos, pero asegurando que la infraestructura (broker, etc.) aguante la carga. MQTT es bastante ligero y podría manejar cientos de nodos con mensajes esporádicos sin problema en una máquina modesta.

Conclusión Final: LoRaIM sienta las bases de un sistema de mensajería IoT sobre LoRa funcional y modulable. Las pruebas confirmaron que los objetivos iniciales se cumplieron, conectando efectivamente un **mundo físico de sensores** con un **mundo digital web**. Las mejoras propuestas abrirían el camino para convertir este prototipo en una solución más completa, segura y aplicable a casos reales de uso. El conocimiento adquirido durante el desarrollo, desde la programación de bajo nivel en MicroPython hasta la coordinación de servicios en contenedores, es de gran valor y puede aplicarse en futuros proyectos IoT de arquitectura similar.

Anexos

Manual de Instalación

A continuación se describen los pasos necesarios para desplegar el sistema LoRaIM tanto del lado del servidor (backend + frontend + broker) como del lado de los dispositivos (nodos LoRa Heltec). Se asume un entorno de trabajo con conectividad a Internet para descargar imágenes Docker y herramientas básicas de desarrollo para interactuar con las placas.

Requisitos previos:

- Dos o más placas **Heltec WiFi LoRa 32 (V3)** con sus antenas LoRa correctamente conectadas (¡importante conectar la antena antes de transmitir para no dañar el módulo RF!).
- Un ordenador (PC) con **Docker** y **Docker Compose** instalados, o alternatively Python 3.10+ si se quisiera ejecutar el backend sin Docker.
- Un entorno para flashear MicroPython en las Heltec y copiar archivos (por ejemplo, el IDE **Thonny** o la herramienta **ampy**, y los drivers USB-UART para la ESP32S3).
- Conexión WiFi local donde operarán los Heltec (se necesita conocer el SSID y contraseña).
- (Opcional) Conocimientos básicos de MQTT por si se quiere probar la comunicación con herramientas como *MQTT Explorer* durante la instalación.

1. Preparar las placas Heltec con MicroPython:

Si las Heltec no tienen MicroPython instalado, es necesario flashearles el firmware. Se puede obtener la última versión estable de MicroPython para ESP32S3 (con soporte para PSRAM si la placa lo tiene) desde el sitio oficial de MicroPython. Luego, usando una herramienta como `esptool.py` se graba el firmware .bin en la placa:

```
esptool.py --chip esp32s3 --port /dev/ttyUSB0 erase_flash
esptool.py --chip esp32s3 --port /dev/ttyUSB0 --baud 115200 write_flash -z 0x0
~/ESP32_GENERIC_S3-20250415-v1.25.0.bin
```

(Reemplazar `/dev/ttyUSB0` por el puerto serial correspondiente y el nombre del binario según la versión descargada.)

Alternativamente, Thonny IDE puede flash automáticamente MicroPython seleccionando la placa ESP32. Tras esta operación, abrir Thonny, seleccionar intérprete MicroPython (ESP32) en el puerto correspondiente y verificar que se tiene acceso a la REPL de MicroPython en la placa.

2. Configurar y cargar el firmware del *LoRaIM Bridge*:

Elegir una de las placas Heltec para que sea el puente. Conectar esa placa vía USB al PC. En Thonny (o mediante `ampy`), crear un archivo llamado `.env` en el sistema de archivos de la placa con el siguiente contenido (basado en `uPython/Heltec_V3_uP_Bridge_Node/.env.example`):

```
# Wi-Fi
WIFI_SSID=NombreDeRedWi-Fi
WIFI_PASSWORD=ContraseñaWi-Fi

# MQTT
MQTT_HOST=XXX.XXX.XXX.XXX
MQTT_PORT=1883
```

```
# Topics (opcionalmente se pueden cambiar, pero mantener por defecto)
MQTT_TOPIC_DOWN=lorachat/down
MQTT_TOPIC_UP=lorachat/up

# LoRa - parámetros que deben igualar a los de los nodos finales
FREQUENCY=868.1
BANDWIDTH=125.0
SPREADING_FACTOR=7
CODING_RATE=5
SYNC_WORD=0x34
TRANSMIT_POWER=14

# Pantalla
BRIGHTNESS=50 # brillo de OLED (0 a 255, opcional ajustar)

# QoS/Redundancia
MQTT_QOS=1
MQTT_RETAIN_UP=0
MQTT_RECONNECT_MAX=30000
```

En `WIFI_SSID` y `WIFI_PASSWORD` coloque las credenciales de la red WiFi local a la que conectará el Bridge. En `MQTT_HOST`, se debe poner la dirección IP del broker MQTT. Si planea ejecutar el broker en el mismo PC host, se puede averiguar su IP en la red local (por ejemplo 192.168.1.100) e insertarla. (También podría usarse un nombre de host si hay DNS local, o usar `MQTT_HOST=mosquitto` solo funcionaría si la Heltec estuviera dentro del mismo docker network, lo cual **no** es el caso; por tanto, es preferible siempre una IP accesible desde la WiFi). Los demás parámetros pueden dejarse como en el ejemplo o ajustarse si fuera necesario (por ejemplo, si en su país la frecuencia LoRa permitida es distinta, o para cambiar potencia transmitida).

Ahora, copiar a la placa todos los archivos de la carpeta `uPython/Heltec_V3_uP_Bridge_Node/`: son principalmente `main.py` y las librerías `_sx126x.py`, `sx1262.py`, `ssd1306.py`, `simple.py` y posiblemente `network_manager.py` (si existiera, según la versión). Esto se puede hacer arrastrando los archivos al sistema de archivos de la placa en Thonny o con comandos `ampy put`. Asegurarse de copiar también el archivo `.env` creado. Una vez cargados, reiniciar la placa (por ejemplo, con `CTRL+D` en la consola de Thonny o power cycle). La placa Bridge debería arrancar el script automáticamente (porque `main.py` se ejecuta al boot). En la consola de Thonny, observará mensajes de conexión (o en la OLED: obtendrá IP y mostrará "MQTT: Connecting..."). Si todo está correcto, debería lograr conectarse a la WiFi (la OLED mostrará la IP asignada) y luego al broker MQTT (OLED: "MQTT: Connected"). Si el broker aún no está arriba, la placa intentará repetidamente hasta conectarse; en tal caso, la OLED puede mostrar "MQTT: Offline" y en la consola se verán intentos de reconexión.

3. Configurar y cargar el firmware de los nodos LoRa de prueba:

Tome las otras placa(s) Heltec que funcionarán como nodos finales. En cada una, se puede usar el firmware de prueba. Se recomienda utilizar la versión mejorada en `uPython/Heltec_V3_uP_Test_Node_v2/` a menos que se desee un comportamiento distinto. Este firmware no requiere un archivo de configuración `.env` ya que los parámetros están definidos dentro de `main.py`. Sin embargo, puede personalizar dentro de `main.py` algunos valores si lo desea, por ejemplo:

- El **nombre** que enviará en los mensajes: por defecto usa `"LoRaTester"` como `from`. Se podría editar esa cadena para diferenciar nodos (ej: "SensorNorte" vs "SensorSur").

- El intervalo `PAUSE` entre pings automáticos (100 s por defecto en v2, puede reducirse a, digamos, 30 s o cualquier valor; evitar valores demasiado bajos que violen duty cycle, sobre todo si `TRANSMIT_POWER` es alta).
- Los pines y parámetros de radio si por alguna razón su hardware difiere (en Heltec V3 estándar no es necesario cambiarlos).

Conecte cada placa de prueba al PC y copie los archivos de `Heltec_V3_uP_Test_Node_v2/` (o v1 si prefiere) hacia ella: particularmente `main.py` y las librerías `_sx126x.py`, `sx1262.py`, `ssd1306.py`. No olvide la librería `ssd1306.py` para la OLED y las de SX1262, pues sin ellas el script no correrá. Una vez cargado, reinicie la placa. En la consola deberá ver el texto "LoRa Tester" indicando inicio, y a partir de ahí cada cierto tiempo mensajes "TX ... Ping ...". En la OLED de ese nodo, la primera línea será "LoRa Tester" y después del primer envío verá "TX 0: Hello Tester" (envío inicial).

Si tiene varias placas, repita este paso en cada una. Asegúrese que todas las placas de prueba tienen los **mismos parámetros LoRa** (frecuencia, SF, etc.) que el puente (los de fábrica en nuestro código ya coinciden con el .env por defecto del puente: 868.1 MHz, SF7, BW125, CR4/5). De lo contrario, no podrán comunicarse.

(Opcional: Si dispone de una única placa Heltec en total, puede simular algunos comportamientos iniciando solo el Bridge y viendo su salida, pero para una comunicación completa se requieren al menos dos nodos.)

4. Desplegar el backend, broker y frontend:

En el PC servidor, ubíquese en el directorio del proyecto (que contiene `docker-compose.yml`). Si Docker está instalado correctamente, ejecute:

```
docker-compose build
docker-compose up -d
```

Esto construirá las imágenes Docker del backend FastAPI y del frontend Flask (basadas en Python 3 slim) e iniciará los contenedores junto con Mosquitto. La opción `-d` los ejecuta en background. Puede verificar con `docker-compose ps` que los tres servicios estén "Up". Mosquitto expone el puerto 1883 en el host; FastAPI en el 8000; Flask en el 5000.

- **Comprobar broker:** Para confirmar que Mosquitto está corriendo, puede usar un cliente MQTT. Por ejemplo, con el comando:

```
mosquitto_sub -h 127.0.0.1 -p 1883 -t "#" -v
```

- **Comprobar backend:** Puede verificar si la API FastAPI responde. Por ejemplo abra en un navegador `http://localhost:8000/docs`, debería cargar la documentación interactiva Swagger de la API. Allí verá los endpoints `/messages/`, `/publish/` y el websocket (no mostrable en Swagger, pero los REST sí). Si hace click en `/messages/` -> "Try it out" -> Execute, debería devolver un 200 OK con un JSON conteniendo probablemente cero mensajes (si aún no llegó nada). Esto indica que el backend está bien levantado.
- **Comprobar frontend:** Abra un navegador web en la dirección `http://localhost:5000`. Debería cargar la interfaz de chat. Inicialmente estará vacía (o puede mostrar mensajes de prueba si ya se intercambiaron antes y quedaron retenidos en la ejecución actual del backend). Verifique que en la consola del navegador (herramientas de desarrollador) no haya errores de JavaScript conectándose al WebSocket. Debería ver en la esquina o título algún indicador de conexión (por ejemplo, el título "LoRa Chat" podría tener un puntito verde si definimos estilo `.online`). Si todo va bien, la interfaz estaría lista esperando datos.

5. Puesta en marcha de la red LoRaIM:

Con el backend y broker activos, y las placas configuradas, encienda primero la placa Bridge (si no lo estaba ya). En su OLED debería confirmar WiFi conectada (IP mostrada) y luego MQTT conectado. Si la IP del broker en .env fue correcta, esto ocurrirá en segundos; de lo contrario, la pantalla del Bridge seguirá indicando reconexiones (revise que `MQTT_HOST` en .env del Bridge sea la IP del host del broker). Luego encienda los nodo(s) de prueba. En la OLED de cada uno debería aparecer "LoRa Tester" y tras unos segundos emitir su primer mensaje (Hello o Ping).

Vaya al navegador web en la página de chat. Si todo está correcto, en el panel de chat **aparecerán** los mensajes que envíen los nodos de prueba. Por ejemplo, debería verse un mensaje "LoRaTester: Hello Tester" o "LoRaTester: Ping 1" inicial. Esto indicará que la comunicación LoRa → Bridge → MQTT → Backend → Frontend ha funcionado. Intente ahora enviar usted un mensaje desde la interfaz: escriba algo en el campo de texto y pulse *Enviar*. Debería inmediatamente verlo reflejado como "Yo: ...". Si el nodo de prueba está en rango y activo, a los pocos instantes ese nodo lo recibirá y (si es firmware de prueba) contestará con ACK, que a su vez aparecerá en la interfaz. ¡El chat LoRaIM está operativo!

6. Solución de problemas comunes:

- *No aparece nada en la web:* Verifique en la consola del navegador si el WebSocket logró conectarse (debe conectarse a `ws://:8000/ws`). Si está en entorno remoto, quizá necesite ajustar la URL en `chat.js` o abrir puertos en firewall. En pruebas locales en la misma máquina, localhost funciona. Si accede desde otro dispositivo (ej. su móvil en la misma red), use la IP local de su PC:5000 para la web, pero entonces el WS intentará :8000, lo cual debería andar si no hay firewall.
- *El Bridge no conecta a WiFi:* Compruebe que SSID y contraseña en .env son correctos. Si la red tiene *captive portal* o es de 5 GHz, la ESP32 no podrá conectar. Use una red WiFi 2.4 GHz convencional. Puede probar el firmware Bridge conectando via Thonny para ver mensajes de debug.
- *El Bridge no conecta a MQTT:* Si WiFi está ok pero broker no, asegúrese de la IP. Pruebe desde una máquina en esa WiFi hacer `ping <IP del broker>` para ver si es alcanzable. Si el broker (docker) está en el mismo PC, asegúrese que el PC tiene IP en la misma subred WiFi y use esa IP (no 127.0.0.1, pues para la placa 127.0.0.1 sería sí misma, no el broker). Otra prueba: instale un cliente MQTT en el móvil o laptop en WiFi e intente conectar a :1883 para descartar problemas de firewall/puerto.
- *Mensajes LoRa no se reciben:* Compruebe que los parámetros LoRa coinciden exacto. Si cambió frecuencia, SF o sync word en un lado, todos deben tener los mismos. También compruebe antenas y distancia; para la primera prueba colóquelos cerca. Si hay dos bridges encendidos por error, podría haber colisiones; asegúrese de tener solo un Bridge.
- *Docker Compose ports conflicts:* Si ya tenía algo en puertos 5000 u 8000, podría haber conflicto. Modifique `docker-compose.yml` para usar puertos libres.
- *Uso sin Docker:* Si no puede usar Docker, alternativamente instale Mosquitto manualmente, instale dependencias de `api/requirements.txt` y ejecute `uvicorn main:app` en `api/app`, y `python app.py` en `frontend/`. Luego configure la IP del broker en el entorno del backend. Sin Docker, hay que manejar CORS en el navegador o ajustar en FastAPI los orígenes permitidos.

Una vez superados estos pasos, tendrá el sistema LoRaIM funcionando. Ya puede pasar a utilizarlo según las indicaciones del manual de usuario. Mantenga las placas dentro de rango y alimentadas. Si desea detener todo, use `docker-compose down` para bajar los servicios de backend, y simplemente apague los nodos.

Manual de Usuario

El sistema LoRaIM está diseñado para ser utilizado de forma muy sencilla a través de una interfaz web de chat. Este manual de usuario explica cómo acceder a la interfaz, cómo enviar y recibir mensajes, y qué información se muestra en pantalla.

Acceso a la interfaz de chat: Para utilizar LoRaIM, abra un navegador web en un dispositivo (PC, portátil, tablet) que esté conectado a la misma red local donde se despliega el servidor. Navegue a la dirección `http://<IP-servidor>:5000` (o el nombre de host, si se configuró). Por ejemplo, si instaló el servidor en un PC con IP 192.168.1.100, escribir `http://192.168.1.100:5000` en la barra de direcciones. Al cargar, debería ver una página titulada "**LoRaIM Chat**" o similar, con un área principal vacía (al inicio) y un recuadro de texto en la parte inferior con un botón "Enviar".

Interfaz y elementos principales:

- En la parte superior de la página suele mostrarse el título del chat. Es posible que haya un indicador de estado de conexión (por ejemplo, un ícono o punto verde) que se ilumina cuando la aplicación está conectada en tiempo real. Esto refleja la conexión WebSocket con el servidor: si por alguna razón se pierde (por ejemplo, el servidor se cae o la red se interrumpe), el indicador cambiará de color o desaparecerá, alertando de que no se están recibiendo actualizaciones en vivo.
- La sección central (el *historial de mensajes*) mostrará las conversaciones en forma de burbujas de chat, similares a las de aplicaciones móviles. Los mensajes que **usted envía** aparecen alineados a la derecha, normalmente con un estilo distinto (por ejemplo, color de fondo diferente) y precedidos por la etiqueta "**Yo:**" para indicar autoría propia. Los mensajes que provienen de los **dispositivos remotos** aparecerán alineados a la izquierda, y vendrán precedidos por el **nombre de origen** del dispositivo que los envió. En la configuración estándar de LoRaIM, si el dispositivo remoto envía mensajes JSON con un campo "from", ese valor se usará como nombre. Por ejemplo, podría ver "LoRaTester: Ping 1" o "SensorX: Alerta" dependiendo de cómo estén identificados los nodos. Si el sistema no tiene un nombre para el origen (por ejemplo, en acuses "ACK" que no llevan formato JSON con nombre), esos mensajes mostrarán la palabra "desconocido:" como origen.
- Cada mensaje también muestra la hora o un indicador temporal. En la versión actual, los mensajes recibidos muestran la hora local recortada (horas:minutos) al momento en que llegaron al navegador. Esto le da contexto de cuándo fue recibido cada mensaje.
- En la esquina superior derecha del panel de mensajes, puede aparecer un pequeño círculo con un número (contador de no leídos) si llegan mensajes nuevos mientras usted está desplazado hacia arriba leyendo mensajes anteriores. Al hacer scroll hasta abajo del todo, ese contador desaparece (significando que ya no hay mensajes sin leer en la vista).

Envío de mensajes: Para enviar un mensaje a los dispositivos LoRa remotos, haga clic en el campo de texto en la parte inferior de la página y escriba su mensaje (por ejemplo, "Hola, ¿estás ahí?"). Puede presionar el botón **Enviar** (o la tecla **Enter** si su navegador lo permite, o **Ctrl+Enter** si se implementó como atajo para nueva línea vs enviar). Al enviar, sucederá lo siguiente:

1. Su mensaje aparecerá inmediatamente en la ventana de chat con la etiqueta "Yo:" y el contenido que escribió. Esto confirma que el mensaje ha sido aceptado por la aplicación y se ha enviado al servidor.
2. Internamente, el servidor retransmitirá su mensaje al nodo LoRaIM Bridge vía MQTT, y éste lo emitirá por radio. Usted no verá eso, pero en la pantalla del Bridge (si tuviera acceso físico a ella) se incrementará el contador de mensajes enviados.

3. Si uno o varios nodos LoRa finales reciben el mensaje, es posible que respondan. En el caso de los nodos de prueba proporcionados, cada uno de ellos enviará un mensaje de **ACK** para confirmar la recepción. Estos acuses normalmente aparecerán en el chat como "desconocido: ACK: " (a menos que se hayan programado de otra forma). Por ejemplo, podría ver "desconocido: ACK:1" poco después de su mensaje. Aunque diga "desconocido", implícitamente representa que el nodo remoto confirmó. Si hubiera múltiples nodos, todos podrían responder; vería múltiples ACK, posiblemente.
4. Si el nodo remoto también desea contestar con texto o envía periódicamente un mensaje, esos aparecerán igualmente. Puede simplemente continuar la conversación enviando otro mensaje cuando guste.

Recepción de mensajes: Los mensajes entrantes de los dispositivos se mostrarán en tiempo real sin que usted tenga que hacer nada. Si la ventana del navegador está abierta, el WebSocket se encarga de "escuchar" y en cuanto llega un nuevo mensaje, se agrega a la conversación. Si por algún motivo su navegador pierde la conexión (verifique el indicador de estado), puede recargar la página; al hacerlo, la aplicación cargará del servidor los últimos ~50 mensajes almacenados para que no pierda contexto mientras estuvo desconectado. Tenga en cuenta que si cierra la página y la vuelve a abrir más tarde, solo verá el historial reciente (no conserva absolutamente todos los mensajes antiguos, a menos que se implemente persistencia en un futuro).

Interpretación de la información en mensajes remotos: Cuando un nodo LoRa envía un mensaje, puede incluir en el texto información adicional como lecturas de sensores, estado, etc., dependiendo de la aplicación. En nuestro nodo de prueba, los mensajes *Ping* no contienen datos especiales, solo son para verificar comunicación. Pero un uso práctico podría ser, por ejemplo, un nodo que envíe "Temperatura: 21.5°C". Este texto se mostrará tal cual después del nombre de origen. Si ese nodo estuviera identificado como "SensorExterior", vería "SensorExterior: Temperatura: 21.5°C". Así sabría quién lo envía y qué indica. Los acuses de recibo "ACK" enviados por el firmware de prueba no tienen más significado que confirmar llegada; el número que llevan es secuencial local del nodo (sirve para debug, pero usted puede interpretarlo simplemente como "ACK recibido").

Limitaciones actuales para el usuario: Es importante señalar que el sistema está pensado para mensajes cortos. No se recomienda enviar textos muy largos o con caracteres especiales fuera de ASCII básico, ya que podría haber limitaciones (LoRa puede manejar unas pocas docenas de bytes por paquete de forma confiable). Tampoco se pueden enviar imágenes, audio ni adjuntos – es solo mensajería de texto plano. Asimismo, actualmente no existe una notificación sonora ni pop-up en la aplicación web, así que debe prestar atención visual a la llegada de nuevos mensajes (o implementar en el navegador algún aviso manual si lo desea).

Uso de múltiples ventanas o usuarios: Si dos personas diferentes acceden a la interfaz web al mismo tiempo desde dos dispositivos, ambas verán el mismo chat (ya que es una sala única unificada). Si una envía un mensaje, en la pantalla de la otra persona **también aparecerá como "Yo:"**, lo cual puede ser confuso porque ambas interfaces tratan los mensajes de origen "sent" como propios. Esto es una limitación actual: básicamente todos los usuarios web se presentan como uno solo. Por lo tanto, se recomienda que en un momento dado use la interfaz web **un solo operador** o, si la usan varios, estén coordinados sabiendo que no podrán distinguir quién escribió qué desde la web. En entornos de prueba esto no es problema, pero no está pensado como chat multiusuario típico de varias personas (a menos que se extienda la funcionalidad en el futuro).

Monitorización del estado de la red: El usuario web no tiene un panel dedicado de control de dispositivos, pero puede inferir algunas cosas:

- Si de repente deja de recibir pings esperados o respuestas, y el indicador de conexión sigue verde, podría ser que el nodo remoto esté fuera de servicio o fuera de alcance. En ese caso, el Bridge podría eventualmente enviar un "offline" si se desconectó (aparecería como mensaje). Si no hay tal mensaje, es posible que el nodo remoto siga conectado pero simplemente no envía nada (por ejemplo, si es un sensor que solo transmite bajo ciertas condiciones).
- Si el indicador de conexión en la web se vuelve rojo/gris (offline), significa que el enlace con el servidor se perdió. Posiblemente el servidor se cayó o hubo un problema de red. En tal caso, sus mensajes "Yo" quizás no estén siendo enviados realmente. Tendrá que esperar a que se restablezca (la página intentará reconectar automáticamente). Puede refrescar la página después de unos segundos para intentar reconectar más rápido.
- En la pantalla del Bridge (si tiene acceso físico), se puede leer la IP y saber si está conectado al MQTT. Esto obviamente no está visible desde la interfaz web, pero es algo que un operador técnico podría revisar in-situ si sospecha fallos.

Cierre de sesión y mantenimiento: No hay un botón de "salir" dado que no hay login; simplemente cierre la pestaña del navegador cuando termine. Los dispositivos LoRa pueden permanecer encendidos enviando datos continuamente. El servidor backend igualmente puede mantenerse en ejecución. Si planea apagar el sistema temporalmente, puede hacer `docker-compose down` en el servidor y apagar las placas. Al volver a encender todo, recuerde que el historial de chat se reiniciará (los viejos mensajes ya no estarán salvo que los nodos los retransmitan o se hayan quedado en el buffer MQTT). Es aconsejable quizá reiniciar también las placas en conjunto con el backend para empezar limpio cada sesión de pruebas.

Consejos de uso:

- Ubique las antenas de los dispositivos de forma vertical y lo más despejada posible para maximizar alcance. Evite pegarlos a superficies metálicas que hagan de pantalla.
- Si hace pruebas de alcance, dos personas pueden coordinarse: una con el nodo remoto (mirando la OLED para ver si llegan mensajes) y otra con el interfaz web enviando texto cada cierto tiempo.
- Tenga en cuenta la **regulación local**: en Europa, por ejemplo, la banda 868 MHz tiene restricción de tiempo de aire (duty cycle 1% típico). Nuestro firmware respeta en parte esto calculando `minimum_pause` tras cada transmisión. No abuse enviando mensajes en bucle rápido porque podría violar normativas de radio. En un caso real, siempre respete los límites de su región para LoRa.
- La OLED de los nodos y Bridge pueden quemarse si se dejan mucho tiempo con la misma imagen estática. En sesiones largas, es recomendable implementar o activar un salvapantallas (en nuestro firmware no incluimos eso, pero como mejora se podría apagar la OLED tras X minutos de inactividad). Por seguridad, apague los dispositivos cuando no los use.

Código comentado

Frontend

Flask

```
from flask import Flask, render_template, request, jsonify
import os
import requests
```

```
app = Flask(__name__, static_folder="static", template_folder="templates")

API_URL = os.getenv("API_URL", "http://fastapi:8000")

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/messages")
def messages():
    limit = request.args.get("limit", 50)
    resp = requests.get(f"{API_URL}/messages/", params={"limit": limit})
    return jsonify(resp.json())

@app.route("/publish", methods=["POST"])
def publish():
    data = request.get_json()
    msg = data.get("message", "")
    resp = requests.post(f"{API_URL}/publish/", json={"message": msg})
    return jsonify(resp.json())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>LoRaIM</title>
  <link rel="preconnect" href="//{{ request.host }}">
  <link rel="dns-prefetch" href="//{{ request.host }}">
  <!-- Estilos y fuentes -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;700&display=swap"
rel="stylesheet">
</head>
<body>
  <div class="chat-container">
    <header class="chat-header">
      <div class="header-title">
        <div class="logo-container">
          
        </div>
        <span>LoRa Instant Messenger</span>
      </div>
      <span id="unread" class="badge hidden">0</span>
    </header>

    <main id="msgs" class="messages-container"></main>
```

```

<footer class="chat-footer">
  <form id="sendForm" class="send-form">
    <input id="msgInput" type="text" placeholder="Escribe un mensaje..."
autocomplete="off" required>
    <button type="submit">></button>
  </form>
</footer>
</div>

<!-- Script WebSocket -->
<script defer src="{ { url_for('static', filename='js/chat.js') } }"></script>
</body>
</html>

```

chat.js

```

const LOCAL_SOURCE = 'sent';
const PAGE = 50; // carga inicial

/* ----- DOM ----- */
const msgsEl = document.getElementById('msgs');
const badgeEl = document.getElementById('unread');
const formEl = document.getElementById('sendForm');
const inputEl = document.getElementById('msgInput');
const headerEl = document.querySelector('.chat-header');

/* ----- estado ----- */
const seenIds = new Set();
let unread = 0;

/* ----- util ----- */
function addBubble({payload, source, time}){
  const wrap = document.createElement('div');
  wrap.className = 'message ' + (source === LOCAL_SOURCE ? 'sent' : 'received');
  wrap.textContent = (source === LOCAL_SOURCE ? 'Yo' : source) + ': ' + payload;

  const ts = document.createElement('span');
  ts.className = 'time';
  ts.textContent = time;
  wrap.appendChild(ts);

  msgsEl.appendChild(wrap);
}

/* ----- carga inicial (últimos PAGE) ----- */
(async () => {
  const res = await fetch(`/messages?limit=${PAGE}`);
  const {messages} = await res.json();
  messages.forEach(m => {
    const id = m.source + '|' + m.payload;
    if (!seenIds.has(id)){

```

```

        addBubble({
            payload: m.payload,
            source : m.source,
            time   : new Date().toLocaleTimeString().slice(0,5)
        });
        seenIds.add(id);
    }
});
msgsEl.scrollTop = msgsEl.scrollHeight;    // al fondo
inputEl.focus();
})();

/* ----- WebSocket ----- */
const wsUrl = (location.protocol === 'https:' ? 'wss://' : 'ws://') +
    location.hostname + ':8000/ws';
const ws = new WebSocket(wsUrl);

ws.onopen = () => headerEl.classList.add('online');
ws.onclose = () => headerEl.classList.remove('online');

ws.onmessage = e => {
    const {payload, source='?'} = JSON.parse(e.data);
    const id = source + '|' + payload;
    if (seenIds.has(id)) return;
    seenIds.add(id);

    addBubble({
        payload, source,
        time: new Date().toLocaleTimeString().slice(0,5)
    });

    /* auto-scroll y badge */
    const wasBottom =
        msgsEl.scrollHeight - msgsEl.scrollTop - msgsEl.clientHeight < 25;
    if (wasBottom){
        msgsEl.scrollTop = msgsEl.scrollHeight;
        unread = 0;
        badgeEl.classList.add('hidden');
    }else{
        unread++;
        badgeEl.textContent = unread;
        badgeEl.classList.remove('hidden');
    }
};

/* ----- enviar ----- */
formEl.addEventListener('submit', async e => {
    e.preventDefault();
    const txt = inputEl.value.trim();
    if (!txt) return;
    await fetch('/publish', {
        method : 'POST',
        headers: {'Content-Type':'application/json'},

```

```

        body    : JSON.stringify({ message: txt })
    });
    inputEl.value = '';
    msgsEl.scrollTop = msgsEl.scrollHeight;
});

/* atajo Ctrl+Enter */
formEl.addEventListener('keydown', e => {
    if (e.key === 'Enter' && e.ctrlKey) formEl.requestSubmit();
});

/* ----- badge reset al llegar al fondo ----- */
msgsEl.addEventListener('scroll', () => {
    if (msgsEl.scrollHeight - msgsEl.scrollTop - msgsEl.clientHeight < 20){
        unread = 0;
        badgeEl.classList.add('hidden');
    }
});

```

Backend

Fast api

```

import os
import json
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi import WebSocket
from collections import deque
from paho.mqtt import client as mqtt_client
from pydantic import BaseModel
import logging
import asyncio

# Configuración MQTT
MQTT_BROKER = os.getenv("MQTT_BROKER", "mosquitto")
MQTT_PORT = int(os.getenv("MQTT_PORT", 1883))
MQTT_TOPIC_UP = os.getenv("MQTT_TOPIC_UP", "lorachat/up")
MQTT_TOPIC_DOWN = os.getenv("MQTT_TOPIC_DOWN", "lorachat/down")

# Buffer circular de mensajes
RECEIVED = deque(maxlen=100)
clients = []

app = FastAPI()

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,

```

```

        allow_methods=["*"],
        allow_headers=["*"],
    )

class PublishPayload(BaseModel):
    message: str

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
logger.info(f"MQTT Broker: {MQTT_BROKER}")
logger.info(f"MQTT Port: {MQTT_PORT}")
logger.info(f"MQTT Topic: {MQTT_TOPIC_UP}")

# Callbacks MQTT
def on_connect(client, userdata, flags, rc):
    """
    Callback de conexión MQTT. Se llama cuando el cliente se conecta al broker.
    :param client:
    :param userdata:
    :param flags:
    :param rc:
    :return:
    """
    client.subscribe(MQTT_TOPIC_UP)

def on_message(client, userdata, msg):
    """
    Callback de mensaje MQTT. Se llama cuando se recibe un mensaje en un topic suscrito.
    :param client:
    :param userdata:
    :param msg:
    :return:
    """
    payload_str = msg.payload.decode()
    try:
        data = json.loads(payload_str)
        message = data.get("message", payload_str)
        sender = data.get("from", "desconocido")
    except Exception:
        message = payload_str
        sender = "desconocido"

    RECEIVED.append({
        "topic": msg.topic,
        "payload": message,
        "source": sender
    })
    logger.info(f"Received message: {data} from topic: {msg.topic}")

# Crear cliente MQTT
mqtt = mqtt_client.Client()

```

```

mqtt.on_connect = on_connect
mqtt.on_message = on_message
try:
    mqtt.connect(MQTT_BROKER, MQTT_PORT)
    mqtt.loop_start()
except Exception as e:
    logger.error(f"Failed to connect to MQTT broker: {e}")
    raise

# Endpoints
@app.get("/")
async def root():
    """
    Endpoint de prueba. Devuelve un mensaje de estado.
    :return:
    """
    return {"status": "ok"}

@app.get("/messages/")
async def get_messages(limit: int = 20, offset: int = 0):
    """
    Devuelve 'limit' mensajes *anteriores* al índice 'offset'
    (0 = el más reciente). Sirve para paginación inversa.
    """
    total = len(RECEIVED)
    start = max(total - offset - limit, 0)
    end = total - offset
    msgs = list(RECEIVED)[start:end]
    return {"count": total, "messages": msgs}

@app.post("/publish/")
async def publish_message(payload: PublishPayload):
    """
    Endpoint para publicar un mensaje en el topic MQTT.
    :param payload:
    :return:
    """
    out = {
        "from": "sent",
        "message": payload.message
    }
    json_msg = json.dumps(out)
    mqtt.publish(MQTT_TOPIC_DOWN, json_msg)
    RECEIVED.append({
        "topic": MQTT_TOPIC_DOWN,
        "payload": payload.message,
        "source": "sent"
    })
    return {"published": out}

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

```

```

"""
Endpoint WebSocket para recibir mensajes en tiempo real.
:param websocket:
:return:
"""

await websocket.accept()
clients.append(websocket)
try:
    while True:
        await asyncio.sleep(1)
        if RECEIVED:
            last_msg = RECEIVED[-1]
            await websocket.send_json(last_msg)
except:
    clients.remove(websocket)

```

Firmware uPython

```

"""
LoRaIM Bridge - Heltec V3 (ESP32-S3 + SX1262 + OLED)
=====

```

Este firmware convierte una placa Heltec WiFi LoRa V3 en un puente bidireccional LoRa ↔ MQTT, facilitando la comunicación transparente entre nodos LoRa y una infraestructura MQTT.

Características Principales:

- Configuración Modular: Lee parámetros desde un archivo `.env`, incluyendo ajustes de Wi-Fi, MQTT, LoRa y OLED, facilitando personalización sin modificaciones de código.
- Identificador Único Automático: Genera un `NODE_NAME` único del tipo `"Node-XXXXXX"`, garantizando la identificación automática de cada dispositivo desplegado.
- Inicialización Integrada: Conecta automáticamente a Wi-Fi, inicializa el módulo LoRa SX1262 y muestra información crítica (estado, IP asignada, mensajes transmitidos y recibidos) en la pantalla OLED.

Flujo de Mensajes:

1. MQTT → LoRa
 - Se suscribe al tópico MQTT definido en `MQTT_TOPIC_DOWN` (p. ej., `lorachat/down`).
 - Cada mensaje MQTT recibido se convierte en un JSON estructurado:


```
{"from": NODE_NAME, "message": "texto"}
```
 - Este paquete JSON se transmite por LoRa hacia los nodos en rango.
2. LoRa → MQTT
 - Escucha continuamente los mensajes LoRa entrantes.
 - Al recibir paquetes LoRa válidos, publica su contenido en el topic definido por `MQTT_TOPIC_UP` (p. ej., `lorachat/up`) con calidad de servicio (QoS) 1 y retención configurable.

- Si la conexión MQTT no está disponible, los mensajes entrantes se almacenan temporalmente en un buffer circular (``pending[]``) y se retransmiten automáticamente al restablecerse la conexión.

Robustez y Fiabilidad MQTT:

- QoS Nivel 1: Asegura la entrega fiable de mensajes.
- Keep-alive 30 s: Mantiene activa y supervisada la conexión con el broker MQTT.
- Aviso de caída (Last-Will): Publica un estado "offline" retenido para alertar inmediatamente ante desconexiones inesperadas.
- Reconexión Automática: Implementa un mecanismo de reintentos con espera incremental (back-off exponencial) desde 2 s hasta 30 s máx.
- Buffering inteligente: Publica automáticamente mensajes pendientes tras reconexiones.

Interfaz Visual (OLED):

- Pantalla OLED SSD1306 de 128×64 píxeles, mostrando:
 - Dirección IP obtenida tras arranque y conexión Wi-Fi.
 - Estado en tiempo real de la conexión MQTT.
 - Mensajes enviados y recibidos por LoRa claramente diferenciados.

Personalización Avanzada:

- Parámetros ajustables directamente desde ``.env``:
 - Frecuencia, potencia, ancho de banda, spreading factor (SF), etc.
 - Brillo de la pantalla OLED.
- Extensible mediante las bibliotecas adjuntas:
 - `sx1262.py` (control módulo LoRa SX1262)
 - `ssd1306.py` (control pantalla OLED)
 - `simple.py` (cliente MQTT simplificado)

English Version:

Main Features:

- Modular Configuration:** Reads settings from a ``.env`` file, including Wi-Fi, MQTT, LoRa, and OLED parameters, allowing easy customization without changing the code.
- Automatic Unique Identifier:** Generates a unique ``NODE_NAME`` in the form ``"Node-XXXXXX"`` for automatic device identification.
- Integrated Initialization:** Automatically connects to Wi-Fi, initializes the LoRa SX1262 module, and displays critical status information (connection status, IP address, transmitted/received messages) on the OLED screen.

Message Flow:

1. **MQTT → LoRa**
 - Subscribes to the MQTT topic defined in ``MQTT_TOPIC_DOWN`` (e.g., ``lorachat/down``).
 - Every received MQTT message is converted to a structured JSON payload:

```
{"from": NODE_NAME, "message": "text"}
```

- This JSON payload is then transmitted over LoRa to nodes within range.

2. **LoRa → MQTT**

- Continuously listens for incoming LoRa packets.
- Upon receiving valid LoRa messages, publishes them to the MQTT topic defined by ``MQTT_TOPIC_UP`` (e.g., ``lorachat/up``) with Quality of Service (QoS) level 1 and optional retention.
- If the MQTT connection is unavailable, messages are temporarily stored in a circular buffer (``pending[]``) and automatically retransmitted upon reconnection.

MQTT Reliability and Robustness:

- QoS Level 1:** Ensures reliable message delivery.
- Keep-alive 30s:** Maintains and supervises an active connection to the MQTT broker.
- Last-Will Notification:** Publishes a retained ``"offline"`` status to immediately notify clients of unexpected disconnections.
- Automatic Reconnection:** Implements exponential back-off reconnection attempts starting from 2 seconds up to a maximum of 30 seconds.
- Intelligent Buffering:** Automatically publishes pending messages after reconnection.

OLED Visual Interface:

- Uses a 128x64 pixel SSD1306 OLED display to show:
 - Device IP address after Wi-Fi connection.
 - Real-time MQTT connection status.
 - Clearly differentiated incoming/outgoing LoRa messages.

Advanced Customization:

- Adjustable parameters directly from ``.env``:
 - LoRa frequency, power, bandwidth, spreading factor (SF), etc.
 - OLED brightness.
- Extendable through included libraries:
 - ``sx1262.py`` (LoRa SX1262 module control)
 - ``ssd1306.py`` (OLED display control)
 - ``simple.py`` (Simplified MQTT client)

Licencia y Créditos:

LoRaIM es un proyecto de código abierto bajo la licencia GPL v3.

2025 LoRaIM.

""""

```
# ----- Imports -----
import time, gc, ubinascii, ujson
from machine import Pin, I2C
import network
from simple import MQTTClient
from sx1262 import SX1262
import ssd1306
```

```
# ----- 1. Leer .env -----
def load_env(path="/.env"):
    """
    Carga las variables de entorno desde un archivo `.env` y devuelve un diccionario con los
    valores.
    :param path:
    :return:
    """
    env = {}
    try:
        with open(path) as f:
            for raw in f:
                raw = raw.strip()
                if raw and not raw.startswith("#"):
                    k, v = raw.split("=", 1)
                    env[k.strip()] = v.strip()
    except OSError:
        pass
    return env

def getenv(env, key, cast=str, default=None):
    """
    Obtiene el valor de una variable de entorno, aplicando un tipo de conversión y un valor
    por defecto si no existe.
    :param env:
    :param key:
    :param cast:
    :param default:
    :return:
    """
    try:
        return cast(env[key])
    except (KeyError, ValueError):
        return default

ENV = load_env()

# ----- 2. Pines Heltec V3 -----
LORA_CS, LORA_SCK, LORA_MOSI, LORA_MISO = 8, 9, 10, 11
LORA_RESET, LORA_BUSY, LORA_DIO1 = 12, 13, 14
VEXT, OLED_SCL, OLED_SDA, OLED_RST = 36, 18, 17, 21

# ----- 3. Parámetros de configuración -----
WIFI_SSID = getenv(ENV, "WIFI_SSID", str, "MySSID")
WIFI_PASS = getenv(ENV, "WIFI_PASSWORD", str, "password")

MQTT_HOST = getenv(ENV, "MQTT_HOST", str, "192.168.1.40")
MQTT_PORT = getenv(ENV, "MQTT_PORT", int, 1883)
MQTT_USER = getenv(ENV, "MQTT_USER", str, None)
MQTT_PASS = getenv(ENV, "MQTT_PASSWORD", str, None)
MQTT_TOPIC_UP = getenv(ENV, "MQTT_TOPIC_UP", str, "lorachat/up").encode()
MQTT_TOPIC_DOWN = getenv(ENV, "MQTT_TOPIC_DOWN", str, "lorachat/down").encode()
MQTT_QOS = getenv(ENV, "MQTT_QOS", int, 1)
MQTT_RETAIN_UP = bool(getenv(ENV, "MQTT_RETAIN_UP", int, 0))
```

```

MQTT_RECON_MAX= getenv(ENV, "MQTT_RECONNECT_MAX", int, 30000) # ms

FREQ      = getenv(ENV, "FREQUENCY", float, 866.3)
BW        = getenv(ENV, "BANDWIDTH", float, 250.0)
SF        = getenv(ENV, "SPREADING_FACTOR", int, 9)
CR        = getenv(ENV, "CODING_RATE", int, 5)
SYNC_WORD = getenv(ENV, "SYNC_WORD", lambda x:int(x,0), 0x12)
TX_POWER  = getenv(ENV, "TRANSMIT_POWER", int, 14)

BRIGHTNESS = getenv(ENV, "BRIGHTNESS", int, 200)

# ----- 4. OLED helpers -----
LINE_H, MAX_LINES = 12, 5
_lines = [""] * MAX_LINES
def oled_log(txt):
    """
    Muestra un mensaje en la pantalla OLED y lo imprime en la consola.
    :param txt:
    :return:
    """
    _lines.pop(0); _lines.append(txt)
    oled.fill(0)
    for i,l in enumerate(_lines):
        oled.text(l, 0, i*LINE_H, 1)
    oled.show()
    print(txt)

# ----- 5. Init hardware -----
def init_oled():
    """
    Inicializa la pantalla OLED y la configura.
    :return:
    """
    Pin(VEXT, Pin.OUT, value=0)
    rst = Pin(OLED_RST, Pin.OUT, value=0)
    time.sleep_ms(20); rst.value(1)
    i2c = I2C(0, scl=Pin(OLED_SCL), sda=Pin(OLED_SDA), freq=400_000)
    global oled
    oled = ssd1306.SSD1306_I2C(128, 64, i2c)
    oled.contrast(BRIGHTNESS)

def wifi_connect():
    """
    Conecta a la red wi-Fi y devuelve la dirección IP asignada.
    :return:
    """
    wlan = network.WLAN(network.STA_IF); wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(WIFI_SSID, WIFI_PASS)
        for _ in range(60):
            if wlan.isconnected(): break
            oled_log("Wi-Fi..."); time.sleep_ms(200)
    return wlan.ifconfig()[0] if wlan.isconnected() else "NO-WIFI"

```

```
def init_lora():
    """
    Inicializa el módulo LoRa SX1262 y lo configura con los parámetros especificados.
    :return:
    """
    l = SX1262(1, LORA_SCK, LORA_MOSI, LORA_MISO,
               LORA_CS, LORA_DIO1, LORA_RESET, LORA_BUSY)
    err = l.begin(freq=FREQ, bw=BW, sf=SF, cr=CR, power=TX_POWER, syncWord=SYNC_WORD,
                  blocking=True)
    if err: raise RuntimeError("LoRa init err %d" % err)
    return l

# ----- 6. Node name -----
mac = network.WLAN(network.STA_IF).config('mac')
NODE_NAME = "Node-" + ubinascii.hexlify(mac).decode()[-6:].upper()

# ----- 7. MQTT → LoRa callback -----
def make_downlink_cb(lora):
    """
    Crea un callback para recibir mensajes MQTT y enviarlos por LoRa.
    :param lora:
    :return:
    """
    def _cb(topic, msg):
        """
        Callback para recibir mensajes MQTT y enviarlos por LoRa.
        :param topic:
        :param msg:
        :return:
        """
        try:
            js = ujson.loads(msg)
            txt = js.get("message", "")
        except: txt = msg.decode()
        if not txt: return
        pkt = ujson.dumps({"from": NODE_NAME, "message": txt})
        lora.send(pkt.encode()+b"\n")
        oled_log("TX LoRa: "+txt[:10])
    return _cb

# ----- 8. Buffer de pendientes (lista circular) -----
PENDING_MAX = 200
pending = []
def pend_append(pkt):
    """
    Añade un paquete al buffer de pendientes y elimina el más antiguo si se supera el
    límite.
    :param pkt:
    :return:
    """
    pending.append(pkt)
    if len(pending) > PENDING_MAX:
```

```

        pending.pop(0)
def pend_popleft():
    """
    Elimina y devuelve el primer paquete del buffer de pendientes.
    :return:
    """
    return pending.pop(0)

# ----- 9. Main loop -----
def main():
    """
    Función principal que inicializa el sistema y gestiona la comunicación entre LoRa y
    MQTT.
    :return:
    """
    init_oled();
    oled_log("Booting")

    ip = wifi_connect();
    oled_log(ip)

    lora = init_lora();
    oled_log("LoRa OK")

    oled_log(NODE_NAME)

    cid = ubinascii.hexlify(mac).decode()
    mqttc = MQTTClient(cid, MQTT_HOST, MQTT_PORT, user=MQTT_USER, password=MQTT_PASS,
keepalive=30)
    lw = ujson.dumps({"from": NODE_NAME, "message": "offline"})
    mqttc.set_last_will(MQTT_TOPIC_UP, lw, True, 1)
    mqttc.set_callback(make_downlink_cb(lora))

    backoff = 2000      # ms
    mqtt_ok = False

    while True:
        # — reconexión MQTT —
        if not mqtt_ok:
            try:
                mqttc.connect()
                mqttc.subscribe(MQTT_TOPIC_DOWN, MQTT_QOS)
                oled_log("MQTT OK")
                mqtt_ok = True
                backoff = 2000
                # vaciar pendientes
                while pending and mqtt_ok:
                    pkt = pend_popleft()
                    try:
                        mqttc.publish(MQTT_TOPIC_UP, pkt, MQTT_RETAIN_UP, MQTT_QOS)
                    except OSError:
                        pend_append(pkt)
                        mqtt_ok = False

```

```

        break
    except OSError:
        oled_log(f"MQTT retry...({len(pending)})")
        time.sleep_ms(backoff)
        backoff = min(backoff*2, MQTT_RECON_MAX)
        continue

# — downlink MQTT → LoRa —————
try:
    mqttc.check_msg()
except OSError:
    mqtt_ok = False

# — uplink LoRa → MQTT —————
pkt, st = lora.recv(timeout_en=True, timeout_ms=200)
if st == 0 and pkt:
    if mqtt_ok:
        try:
            mqttc.publish(MQTT_TOPIC_UP, pkt, MQTT_RETAIN_UP, MQTT_QOS)
        except OSError:
            mqtt_ok = False
            pend_append(pkt)
    else:
        pend_append(pkt)
    try:
        js = ujson.loads(pkt)
        oled_log("RX "+js.get("from", "")[-6:]+":"+js.get("message", "")[:8])
    except: oled_log("RX pkt")
if mqtt_ok:
    try: mqttc.ping()
    except OSError: mqtt_ok = False

gc.collect()
time.sleep_ms(50)

if __name__ == "__main__":
    main()

```

7. Bibliografía

- FastAPI Modern Python Web Development. O'REILLY ISBN: 978-1-098-13550-8
- Flask. <https://flask.palletsprojects.com/en/stable/>
- FastApi-MQTT. <https://sabuhish.github.io/fastapi-mqtt/>
- MQTT: <https://mqtt.org/getting-started/>
- MicroPython: <https://docs.micropython.org/en/latest/>
- Docker Compose: <https://docs.docker.com/compose/>
- LoRa Modulation Basics: <https://web.archive.org/web/20190718200516/https://www.semtech.com/uploads/documents/an1200.22.pdf>

